

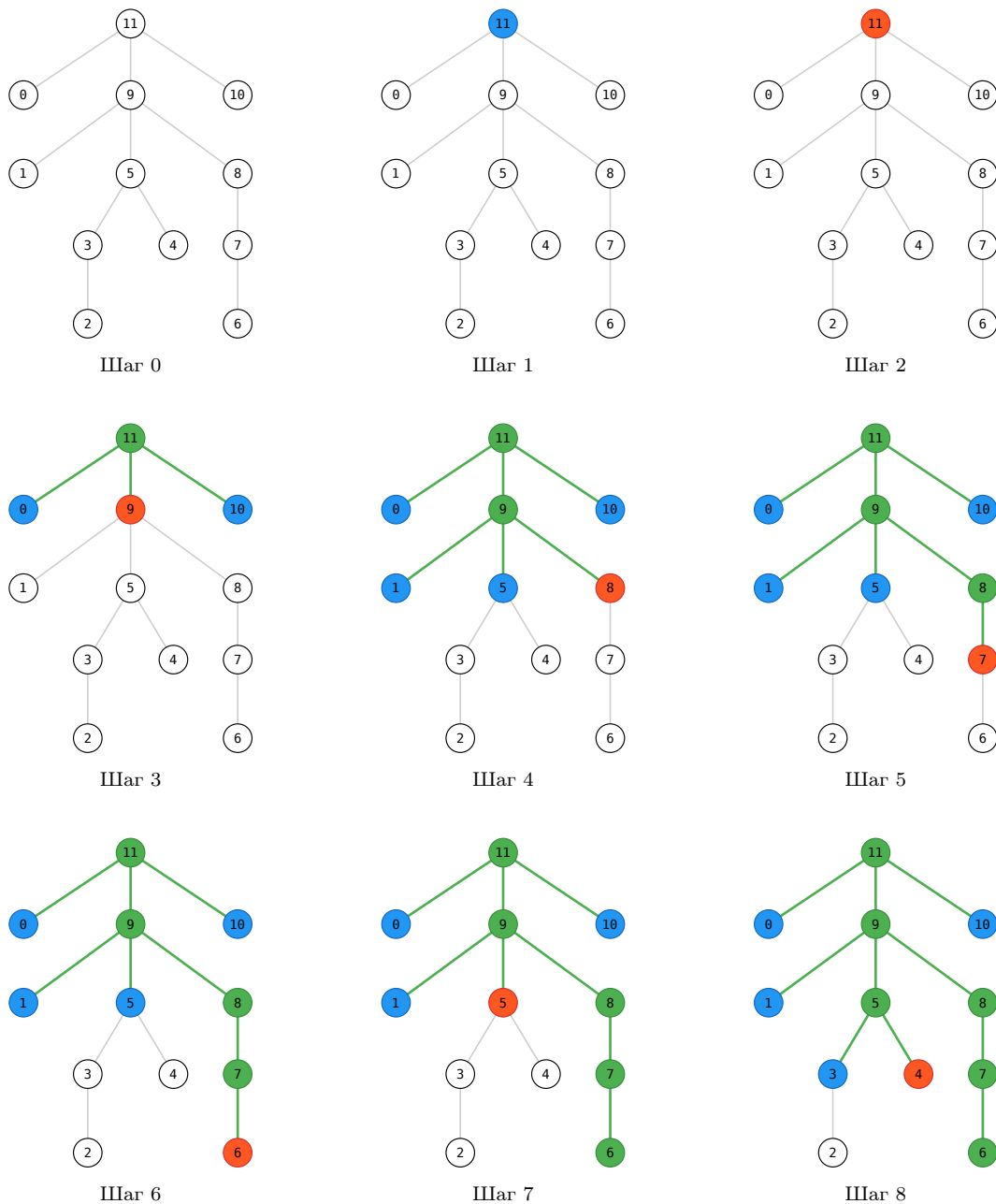
Алгоритмы — КиТГ

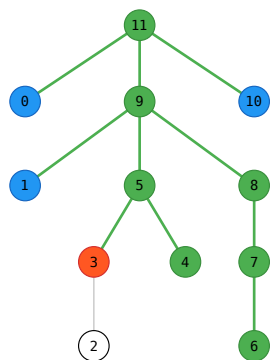
Алгоритм 1. Поиск в глубину (DFS) и поиск в ширину (BFS)

Поиск в глубину (DFS) использует стек: из текущей вершины уходим как можно глубже, и только упёршись в тупик, возвращаемся назад. **Поиск в ширину (BFS)** использует очередь: сначала обрабатываются все соседи текущей вершины, затем соседи соседей — обход идёт «слоями».

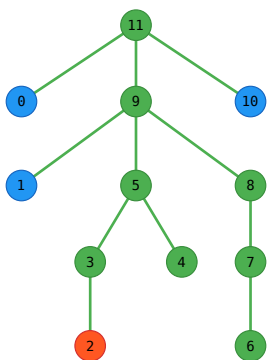
Цветовые обозначения на рисунках: текущая (активная) вершина — оранжевая, уже в стеке/очереди — синяя, обработанная — зелёная; рёбра дерева обхода выделены зелёным.

Пример работы DFS (старт из S):

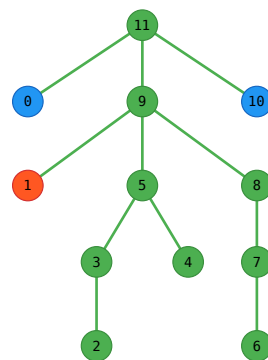




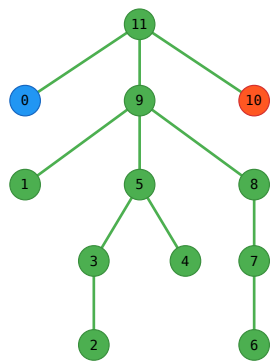
Шаг 9



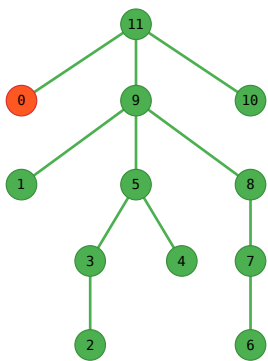
Шаг 10



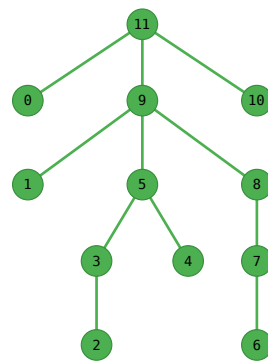
Шаг 11



Шаг 12

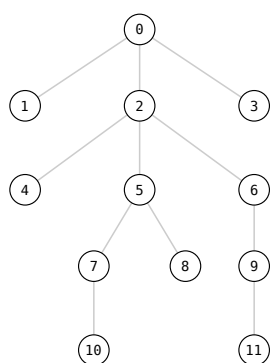


Шаг 13

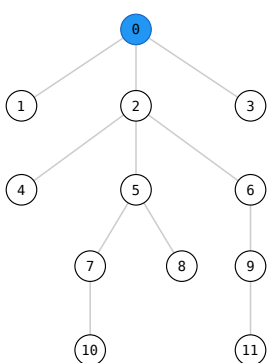


Шаг 14

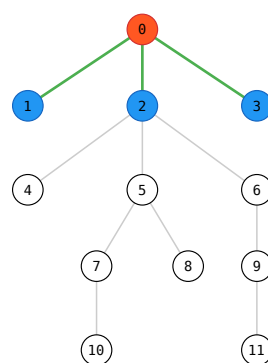
Пример работы BFS (старт из S):



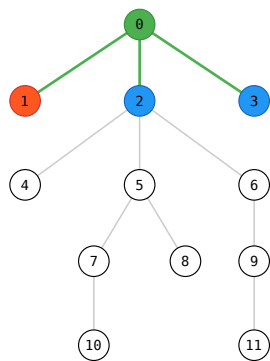
Шаг 0



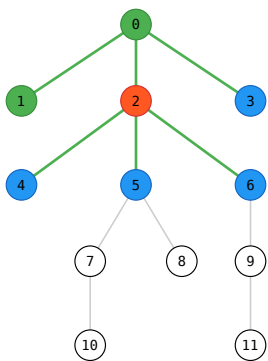
Шаг 1



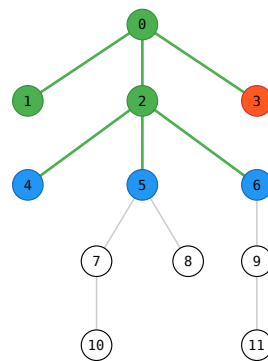
Шаг 2



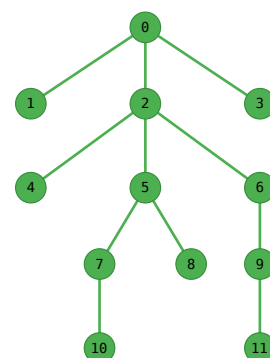
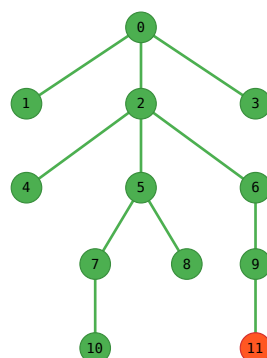
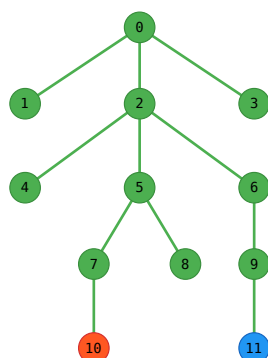
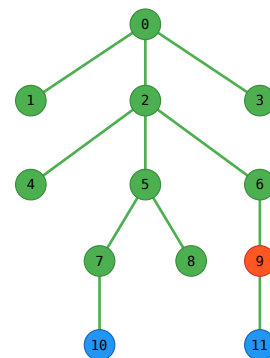
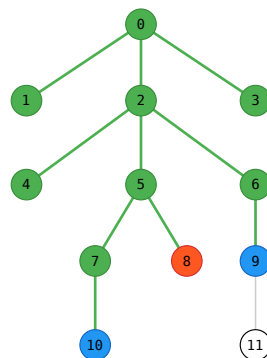
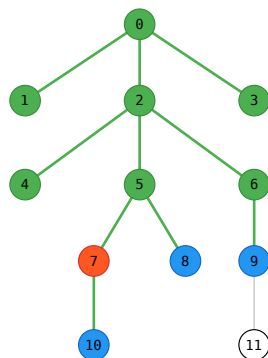
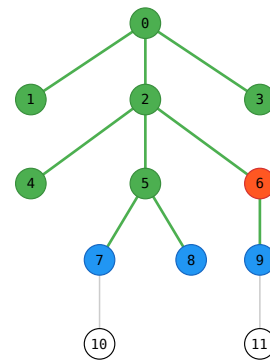
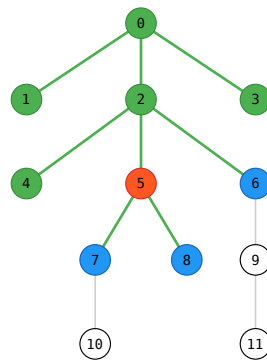
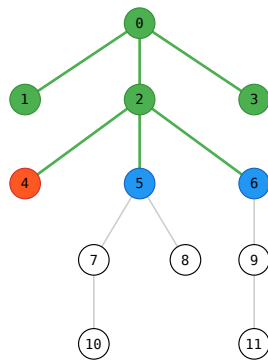
Шаг 3



Шаг 4



Шаг 5



Алгоритм 2. Топологическая сортировка (Topsort)

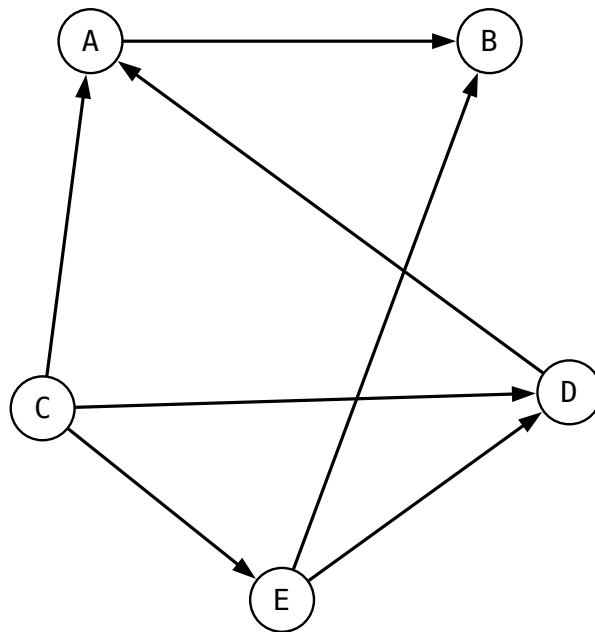
Дано: ориентированный граф без циклов, задающий частичный порядок. Построить: линейный порядок, согласованный с исходным.

Шаг I. Серия поисков в глубину (DFS). Когда вершина становится «тупиковой» (все её исходящие рёбра уже обработаны), она кладётся в стек.

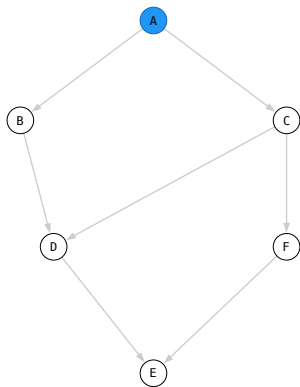
Шаг II. Нумеруем вершины по стеку *снизу вверх*: самая нижняя вершина стека получает номер 1, верхняя — номер n . После этого дорисовываем рефлексивные петли и недостающие рёбра: из каждой вершины с *большим* номером — во все вершины с *меньшим* номером.

Результат — граф, задающий отношение **линейного порядка**.

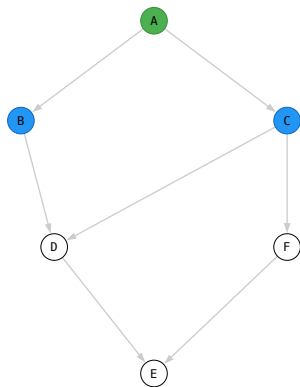
Работа алгоритма по шагам. Исходный граф:



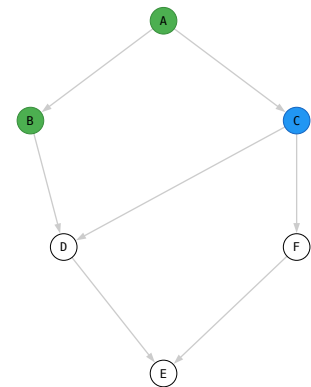
Активная ветка DFS — синяя, текущая вершина — оранжевая, тупиковые (уложенные в стек) — зелёные, рёбра дерева обхода — зелёные. Стек показан справа (верх стека — сверху).



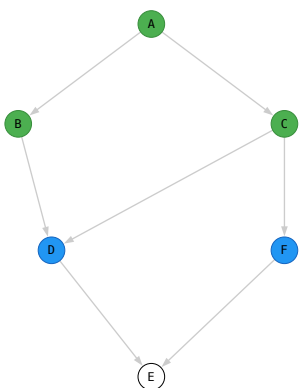
Шаг 0. Исходный граф, стек пуст



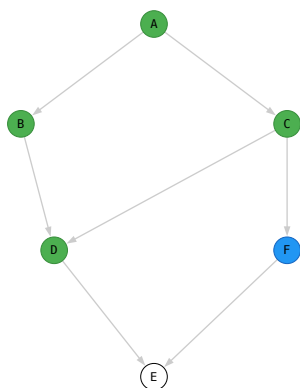
Шаг 1. DFS из A: идём $A \rightarrow B$



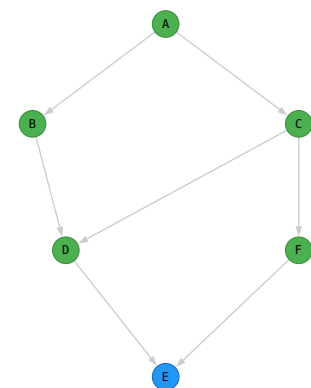
Шаг 2. B тупиковая $\rightarrow$ в стек



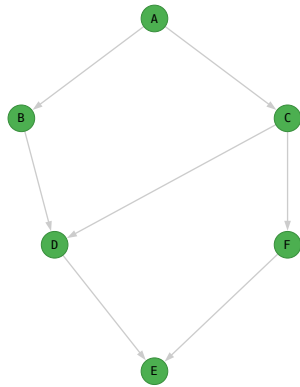
Шаг 3. A тупиковая $\rightarrow$ в стек



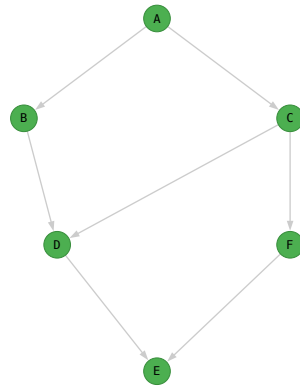
Шаг 4. DFS из C: $C \rightarrow D$; D $\rightarrow$ в стек



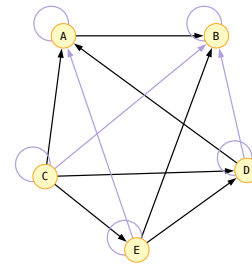
Шаг 5. $C \rightarrow E$; E $\rightarrow$ в стек



Шаг 6. C тупиковая; шаг I завершён



Шаг 7. Нумерация по стеку + петли



Шаг 8. Недостающие рёбра

Стек:
$C = 5$
$E = 4$
$D = 3$
$A = 2$
$B = 1$

Комментарии к шагам. На шаге 1 запускаем DFS из A : единственное исходящее ребро ведёт в B . У B исходящих рёбер нет — она тупиковая, кладём её в стек (шаг 2); возвращаемся в A — все её рёбра обработаны, A тоже уходит в стек (шаг 3). Запускаем DFS из следующей непосещённой вершины C : ребро $C \rightarrow D$ приводит в D , откуда идёт только ребро $D \rightarrow A$ в уже посещённую вершину, поэтому D тупиковая (шаг 4). Далее $C \rightarrow E$: оба ребра из E ($E \rightarrow D$, $E \rightarrow B$) ведут в посещённые вершины — E в стек (шаг 5). Наконец тупиковой становится и C (шаг 6).

На шаге 7 нумеруем вершины по стеку снизу вверх: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$ — и дорисовываем рефлексивные петли (светло-фиолетовые). На шаге 8 добавляем недостающие рёбра из вершин с большими номерами в вершины с меньшими (тоже светло-фиолетовые: $D \rightarrow B$, $E \rightarrow A$, $C \rightarrow B$ и т.д.).

Стек (низ $\rightarrow$ верх): B, A, D, E, C .

Нумерация вершин: $B = 1$, $A = 2$, $D = 3$, $E = 4$, $C = 5$.

Линейный порядок: $B \leq A \leq D \leq E \leq C$.

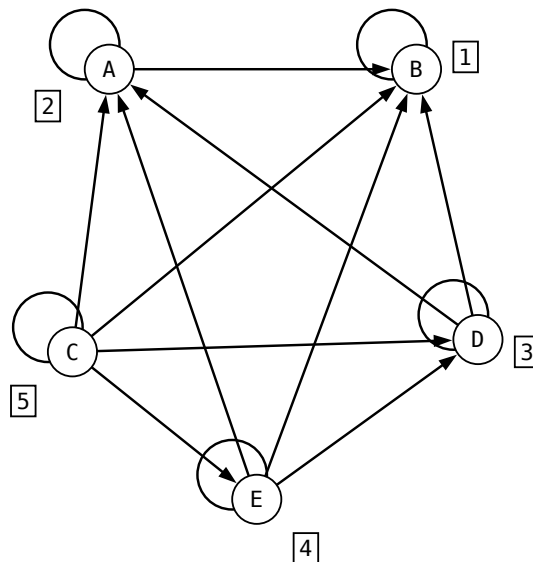


Рис. 1. Результат Topsort — линейный порядок $B \leq A \leq D \leq E \leq C$

Алгоритм 3. Алгоритм Флойда–Уоршелла (построение транзитивного замыкания)

Вход: матрица смежности A_f б.о. f (размер $n \times n$).

Выход: матрица транзитивного замыкания $\tilde{A}$.

Шаг 1: $\forall v \in V$: рассматриваем v как «промежуточную» вершину.

Шаг 2: Для каждой пары (i, j) : если $a_{iv} = 1$ и $a_{vj} = 1$, то полагаем $a_{ij} := 1$.

Повторяем для всех вершин v .

После дорисовывания графа получаем транзитивное замыкание.

Алгоритм 4. Алгоритм поиска мостов

Вход: $G(V, E)$ — связный, неориентированный.

Выход: полный список мостов.

Шаг 1. Серия DFS:

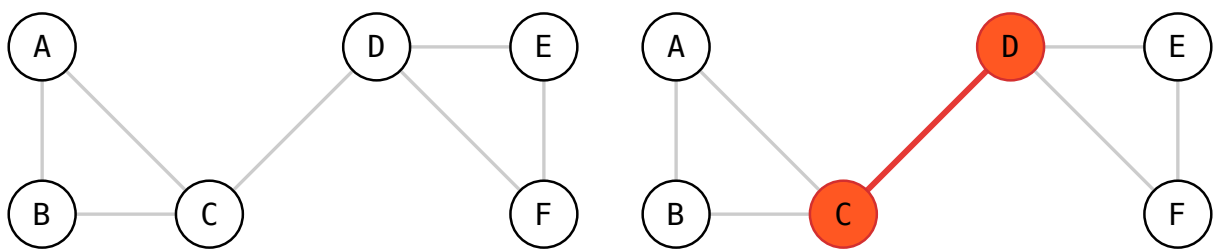
- а) при первом посещении вершины — добавить её в список;
- б) при первом посещении ребра — ориентировать его в направлении обхода.

Шаг 2. Инвертируем все рёбра: $\bar{E}$ — меняем направление каждого ребра на противоположное.

Шаг 3. Серия DFS по списку из шага 1 на графе $G(V, \bar{E})$; каждая серия (компонента) красится своим цветом.

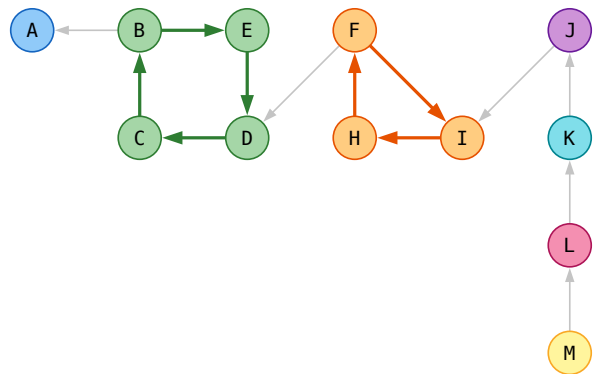
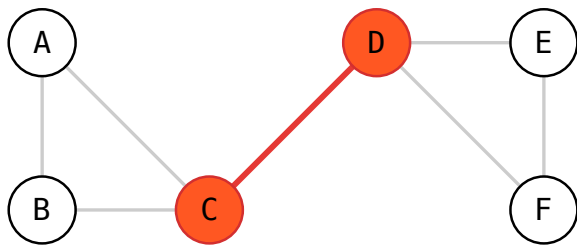
Результат: мосты — это в точности рёбра, концы которых оказались *разного цвета* (в разных компонентах обратного обхода).

Работа алгоритма по шагам (граф из 12 вершин):



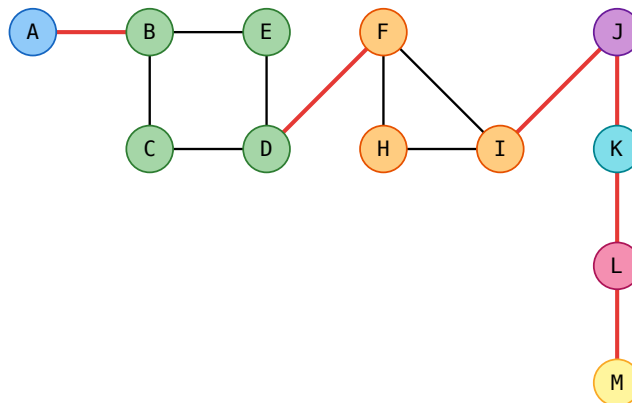
Шаг 0. Исходный граф

Шаг 1. DFS из A: ориентируем рёбра по обходу



Шаг 2. Инвертируем все рёбра

Шаг 3. Серия DFS: компоненты — цветами



Шаг 4. Мосты — рёбра между разными цветами (красные)

Комментарии к шагам. На шаге 1 поиск в глубину из A ориентирует рёбра в порядке обхода: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, ребро $E-B$ впервые посещается из E (ориентируем $E \rightarrow B$); далее $D \rightarrow F \rightarrow H \rightarrow I \rightarrow J \rightarrow K \rightarrow L \rightarrow M$ и ребро $I \rightarrow F$. На шаге 3 серия DFS на инвертированном графе (в порядке списка $A, B, C, \dots$) даёт компоненты $\{A\}$, $\{B, C, D, E\}$, $\{F, H, I\}$, $\{J\}$, $\{K\}$, $\{L\}$, $\{M\}$: из A по инвертированным рёбрам никуда не попасть; из B обходится цикл $B \rightarrow E \rightarrow D \rightarrow C \rightarrow B$; из F — цикл $F \rightarrow I \rightarrow H \rightarrow F$; вершины J, K, L, M остаются поодиночке.

Мосты данного графа (рёбра между разными компонентами): $A-B$, $D-F$, $I-J$, $J-K$, $K-L$, $L-M$.

Алгоритм 5. Алгоритм Косарайю (построение графа Герца)

Алгоритм для выделения компонент сильной связности.

Шаг I. Серия DFS: «тупиковая» вершина (все исходящие рёбра которой уже обработаны) кладётся в стек. Стек заполняется снизу вверх по порядку завершения обработки вершин.

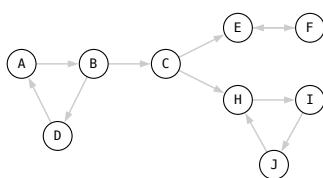
Зачем это нужно: если из компоненты X есть путь в компоненту Y (X «выше по течению»), то хотя бы одна вершина X окажется в стеке *выше* всех вершин Y . Поэтому на шаге III обход стартует из компонент, «выше по течению», и не «перетекает» в чужие компоненты.

Шаг II. $\text{inv } E$ — инвертируем все рёбра.

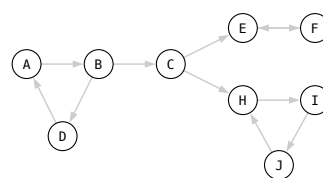
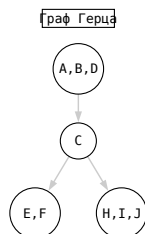
Шаг III. Серия DFS, согласованная со стеком: достаём вершины с верха стека; из каждой ещё не помеченной вершины запускаем DFS по *инвертированным* рёбрам — все достижимые непомеченные вершины образуют одну компоненту сильной связности (красим своим цветом).

Результат: компоненты сильной связности; стянув каждую в одну вершину, получаем граф Герца.

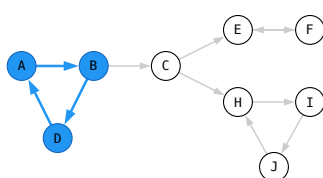
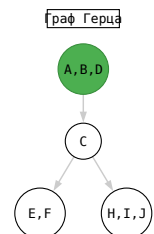
Работа алгоритма по шагам (граф из примера выше; стек показан справа, верх стека — сверху):



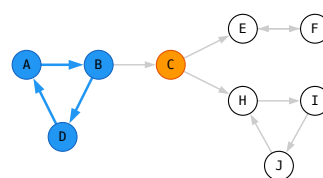
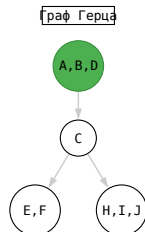
Шаг 0. Исходный граф



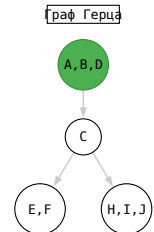
Шаг 1. Серия DFS из A : стек по временам выхода

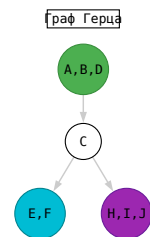
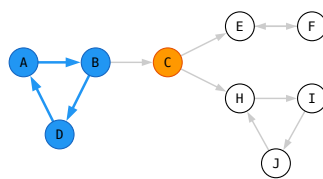
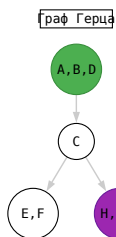
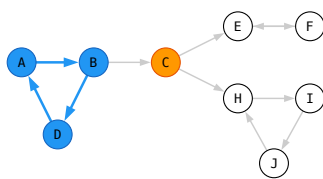


Шаг 2. Инвертируем все рёбра



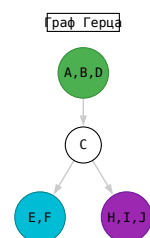
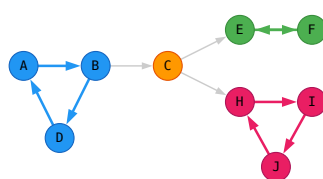
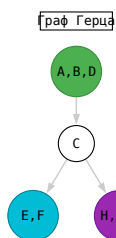
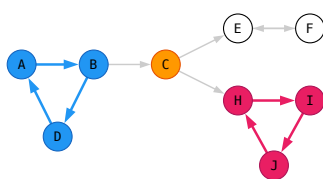
Шаг 3. Сверху стека A : DFS даёт $\{A, D, B\}$





Шаг 4. Следующая непомеченная C : $\{C\}$

Шаг 5. Из H : компонента $\{H, J, I\}$



Шаг 6. Из E : компонента $\{E, F\}$

Шаг 7. Итог: 4 компоненты сильной связности

Комментарии к шагам. На шаге 1 поиск в глубину из A обходит граф: $A \rightarrow B \rightarrow D$ (ребро $D \rightarrow A$ ведёт в посещённую вершину, D — тупиковая, кладём в стек); далее $B \rightarrow C \rightarrow E \rightarrow F$ (ребро $F \rightarrow E$ — в посещённую, F в стек, затем E); $C \rightarrow H \rightarrow I \rightarrow J$ ($J \rightarrow H$ — в посещённую, в стек уходят J, I, H); наконец в стек уходят C, B, A . Стек снизу вверх: $D, F, E, J, I, H, C, B, A$.

На шаге III достаём вершины сверху: из A по инвертированным рёбрам достижимы D и B — компонента $\{A, B, D\}$ (шаг 3); B уже помечена, следующая C — из неё по инвертированным рёбрам можно попасть только в помеченную B , компонента $\{C\}$ (шаг 4); из H достижимы J и I — $\{H, I, J\}$ (шаг 5); из E достижима F — $\{E, F\}$ (шаг 6). Все вершины помечены — алгоритм завершён (шаг 7).

Алгоритм 6. Алгоритм Флёрри

Вход: G — Э или ПЭ.

Выход: эйлеров цикл / эйлеров путь.

P.S. Граф не может быть Э и ПЭ одновременно.

Инициализация:

1. $\exists \Rightarrow \text{start} \in V$ — любая вершина.
2. $\text{П}\exists \Rightarrow \text{start} \in V: \deg(v) \not\equiv 2.$
3. (ориент.) $\Rightarrow \text{start} \in V: \delta^-(v) = \delta^+(v) + 1.$

Шаг (step): на каждом шаге идём по ребру, которое **не является мостом** в оставшемся графе (если есть выбор). По мосту идём, только когда другого ребра из текущей вершины нет. Пройденное ребро удаляем.

Пример.

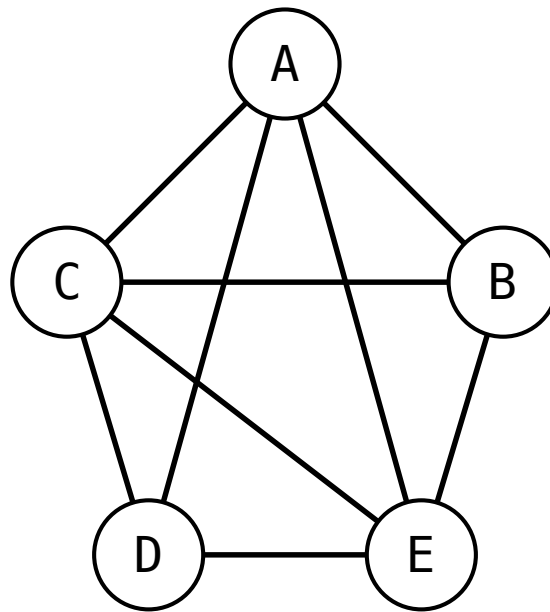


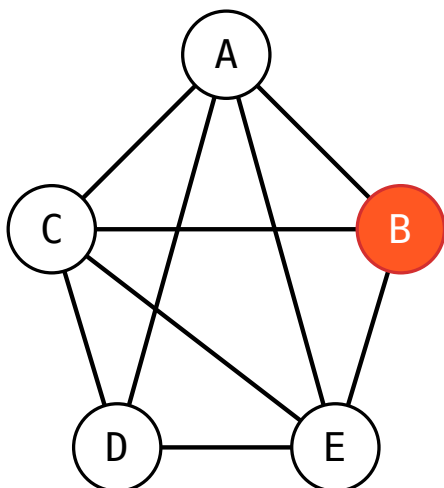
Рис. 2. Алг. Флёрн: пример графа

Степени: $\deg A = \deg C = \deg E = 4$, $\deg B = \deg D = 3$ — ровно две нечётные вершины, граф полуэйлеров; стартуем из нечётной вершины B (закончим в D).

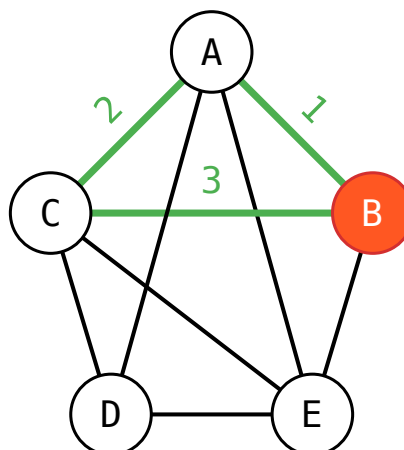
№	вершина	ребро	комментарий (мосты в оставшемся графе)
1	B	$B-A$	мостов нет, берём любое ребро
2	A	$A-C$	мостов нет
3	C	$C-B$	мостов нет
4	B	$B-E$	единственное оставшееся ребро из B
5	E	$E-C$	$E-C$ не мост (цикл $C-D-E$ цел)
6	C	$C-D$	единственное ребро из C (мост, но выбора нет)
7	D	$D-E$	$D-E$ не мост, а $D-A$ — мост: избегаем
8	E	$E-A$	единственное ребро из E
9	A	$A-D$	последнее ребро; финиш в D

Эйлеров путь: $B, A, C, B, E, C, D, E, A, D$ — все 9 рёбер ровно по одному разу, от одной нечётной вершины (B) до другой (D).

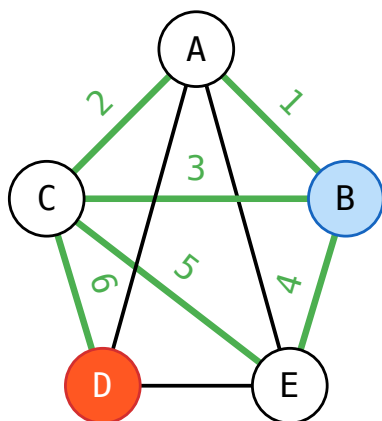
Работа алгоритма по шагам (номера на рёбрах — порядок прохождения; старт B — синий, текущая вершина — оранжевая):



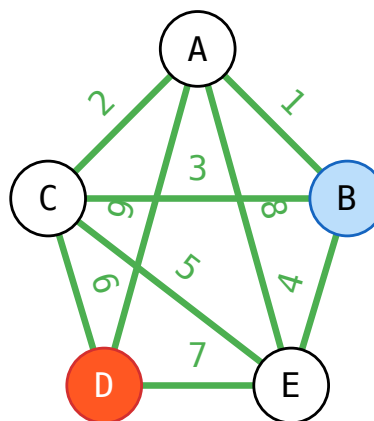
Шаг 0. Старт в нечётной вершине B



Шаги 1–3: $B-A-C-B$



Шаги 4–6: $B-E-C-D$



Шаги 7–9: $D-E-A-D$, финиш в D

Алгоритм 7. Алгоритм поиска эйлерова цикла (на списках инцидентности)

Идея. Граф задан списками инцидентности (для каждой вершины — список исходящих рёбер). Используются стек `temp` и список `result`:

1. кладём стартовую вершину в стек;
2. пока стек не пуст, смотрим на вершину v с верха стека:
 - если в списке v осталось неиспользованное ребро $v \rightarrow u$ — вычёркиваем его из списка и кладём u в стек;
 - если список v исчерпан — снимаем v со стека и дописываем её в `result`;
3. в конце `result`, прочитанный в *обратном порядке*, — эйлеров цикл (путь).

Пример.

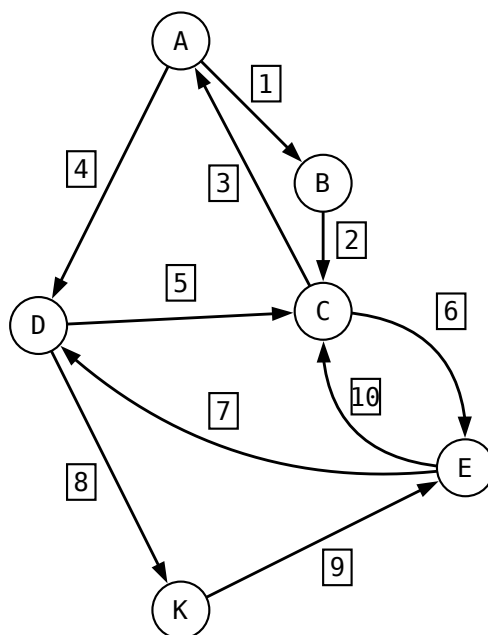


Рис. 3. Алг. на списках инцидентности

Полустепени: у вершины A выходов на один больше, чем входов ($\delta^-(A) = 2$, $\delta^+(A) = 1$), у C — наоборот ($\delta^-(C) = 2$, $\delta^+(C) = 3$), у остальных входы равны выходам — по критерию граф полуэйлеров; эйлеров путь идёт из A в C .

Списки инцидентности (исходящие рёбра):

A : B, D
 B : C
 C : A, E
 D : C, K
 E : D, C
 K : E

Работа алгоритма: сначала стек только растёт — $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow C \rightarrow E \rightarrow D \rightarrow K \rightarrow E \rightarrow C$ (каждый раз вычёркиваем первое неиспользованное ребро из списка текущей вершины). К этому моменту все 10 рёбер израсходованы, и стек начинает разбираться: вершины по очереди исчерпаны и уходят в **result**.

Состояния (**temp** — стек после фазы роста, читается снизу вверх; **result** — порядок выгрузки, сверху вниз):

temp (низ → верх)	result
A	C
B	E
C	K
A	D
D	E
C	C
E	D
D	A
K	C
E	B
C	A

Эйлеров путь — это **result**, прочитанный снизу вверх:

$$A, B, C, A, D, C, E, D, K, E, C$$

— все 10 дуг ровно по одному разу, из A в C .

Алгоритм 8. Построение гамильтонова пути в условиях теоремы Дирака

Инициализация: $v_1, v_2, \dots, v_n$ — произвольный порядок *всех* вершин графа.

Шаг (поворот): пока в последовательности есть несмежная соседняя пара (v_i, v_{i+1}) , ищем индекс $j > i + 1$, для которого *оба* ребра существуют:

$$\exists e_1 = v_i v_j \quad \text{и} \quad \exists e_2 = v_{i+1} v_{j+1};$$

(условие Дирака гарантирует, что такой j найдётся). Переворачиваем отрезок $v_{i+1} \dots v_j$:

$$v_1 \dots v_i \mid \underbrace{v_j v_{j-1} \dots v_{i+1}}_{\text{перевёрнут}} \mid v_{j+1} \dots v_n.$$

На стыках теперь стоят пары (v_i, v_j) и (v_{i+1}, v_{j+1}) — обе смежные, а внутри перевёрнутого отрезка смежность пар не изменилась. Число несмежных пар строго уменьшилось; повторяем, пока их не останется — получился гамильтонов путь.

Пример. Возьмём граф с рёбрами $A-C, C-E, E-F, F-D, D-B, B-A, A-D, C-F, B-E$ (к графу с рис. выше по тексту добавлено ребро $B-E$, чтобы выполнялось условие Дирака: теперь все степени равны $3 = \frac{6}{2} = \frac{n}{2}$).

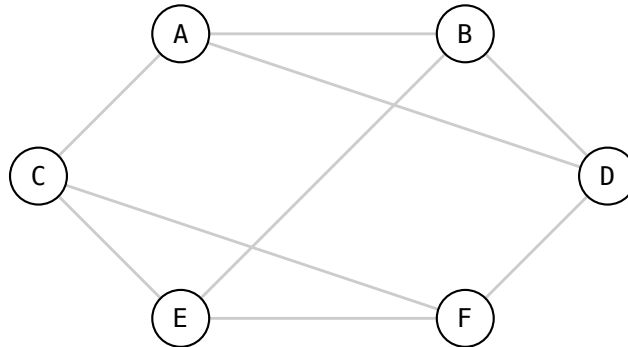


Рис. 4. Алг. Дирака: граф $A, \dots, F$, все степени = 3

Начальный порядок: E, A, B, D, F, C — т.е. $v_1 = E, v_2 = A, v_3 = B, v_4 = D, v_5 = F, v_6 = C$. Порядок *произвольный* (алгоритм стартует с любого); этот выбран для компактности примера — в нём всего одна несмежная соседняя пара. Проверяем пары: (E, A) — *несмежная* (ребра $E-A$ нет); остальные пары $(A, B), (B, D), (D, F), (F, C)$ — смежные.

Чиним пару $(v_i, v_{i+1}) = (v_1, v_2) = (E, A)$. По описанию шага ищем индекс $j > i + 1 = 2$, для которого существуют *оба* ребра $v_i v_j$ и $v_{i+1} v_{j+1}$, т.е. $E \sim v_j$ и $A \sim v_{j+1}$ — перебираем кандидатов $j = 3, 4, 5$ по порядку:

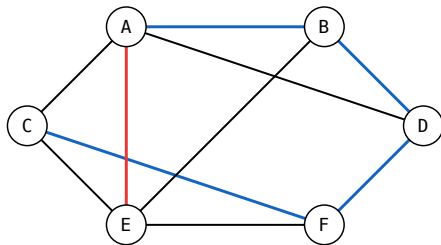
- $j = 3$: $v_3 = B$, $v_4 = D$. Проверяем: $E \sim B$? — да (ребро $B-E$); $A \sim D$? — да (ребро $A-D$). **Оба условия выполнены** — берём $j = 3$ (далее можно не перебирать; для сравнения: $j = 4$ не подошёл бы — уже первое условие $E \sim v_4 = D$ не выполнено, ребра $E-D$ в графе нет).

Переворачиваем отрезок $v_{i+1} \dots v_j = v_2 \dots v_3 = (A, B)$:

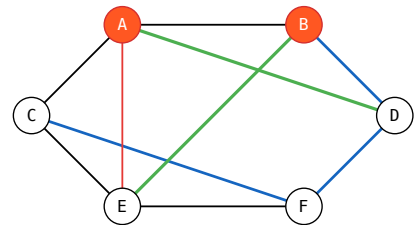
$$E, \underline{A}, \underline{B}, D, F, C \longrightarrow E, \underline{B}, \underline{A}, D, F, C.$$

На стыках перевёрнутого отрезка встали проверенные рёбра: слева — $v_1 v_3 = E-B$, справа — $v_2 v_4 = A-D$. Все соседние пары теперь смежные: $E-B$, $B-A$, $A-D$, $D-F$, $F-C$ — это **гамильтонов путь**. Более того, концы E и C тоже смежны ($C-E$ — ребро), так что путь замыкается в **гамильтонов цикл** E, B, A, D, F, C, E .

Работа алгоритма по шагам (синие рёбра — смежные соседние пары, красное — несмежная пара, зелёные — рёбра поворота и итоговый путь):



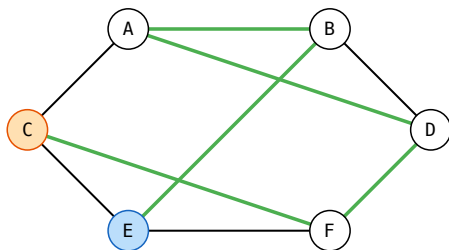
Последовательность: E, A, B, D, F, C
Несмежная пара: (E, A)



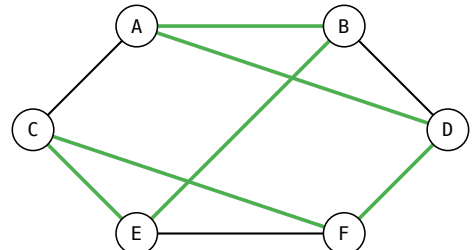
Поворот: $j = 3$, $v_j = B$, $v_{j+1} = D$
 $E-B$ и $A-D$ есть в графе $\rightarrow$ переворачиваем (A, B)

Шаг 0. Порядок E, A, B, D, F, C ; пара (E, A) несмежная

Шаг 1. Поворот: $j = 3$, рёбра $E-B$ и $A-D$ есть



Гамильтонов путь: E, B, A, D, F, C



C и E смежны $\rightarrow$ гамильтонов цикл

Шаг 2. Отрезок (A, B) перевёрнут: гамильтонов путь

Шаг 3. $C \sim E$: замыкаем в гамильтонов цикл

Алгоритм 9. Код Прюфера (прямая задача)

Дано: $G(V, E)$ — дерево, $|V| = n$, порядок лексикографический.

Выход: $\text{Pruf}(G) = \{v_{i_1}, \dots, v_{i_{n-2}}\}$.

Шаг (step): удаление лексикографически минимального листа; в код записывается его родитель (единственный сосед). Повторяем, пока не останется 2 вершины: код имеет длину $n - 2$.

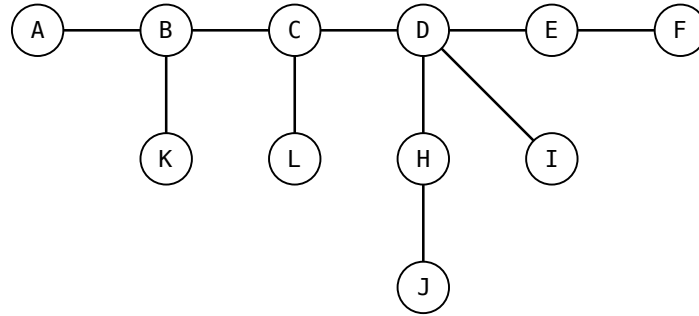


Рис. 5. Код Прюфера: дерево 11 вершин

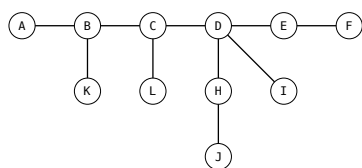
Пример. Дерево (11 вершин): $A-B, B-C, C-D, D-E, E-F, D-H, H-J, D-I, B-K, C-L$. Порядок: $A, B, C, D, E, F, H, I, J, K, L$.

По шагам (лист $\rightarrow$ в код пишем его соседа):

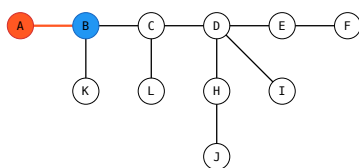
1. листья $\{A, F, I, J, K, L\}$; минимальный A , сосед $B \rightarrow$ код (B) ;
2. минимальный лист F , сосед $E \rightarrow$ код (B, E) ;
3. E стала листом; минимальный лист E , сосед $D \rightarrow (B, E, D)$;
4. минимальный лист I , сосед $D \rightarrow (B, E, D, D)$;
5. минимальный лист J , сосед $H \rightarrow (B, E, D, D, H)$;
6. H стала листом; сосед $D \rightarrow (B, E, D, D, H, D)$;
7. D стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C)$;
8. минимальный лист K , сосед $B \rightarrow (B, E, D, D, H, D, C, B)$;
9. B стала листом; сосед $C \rightarrow (B, E, D, D, H, D, C, B, C)$; остались две вершины C и L — стоп.

$$\text{Pruf}(G) = (B, E, D, D, H, D, C, B, C), \quad |\text{Pruf}(G)| = 11 - 2 = 9.$$

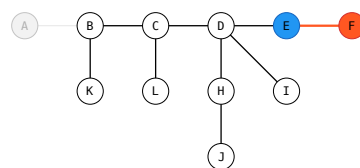
Работа алгоритма по шагам. На каждом шаге удаляется лист с минимальной меткой (оранжевый), в код дописывается его сосед (синий); удалённые вершины и рёбра показаны серым.



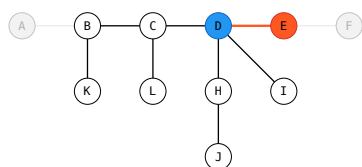
Шаг 0. Исходное дерево



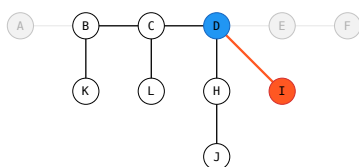
Шаг 1. $A \rightarrow$ код: B



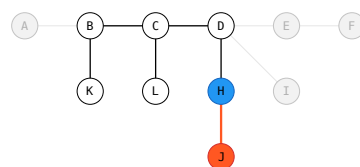
Шаг 2. $F \rightarrow$ код: E



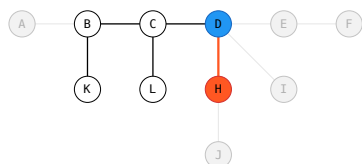
Шаг 3. $E \rightarrow$ код: D



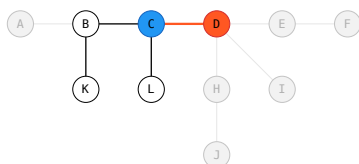
Шаг 4. $I \rightarrow$ код: D



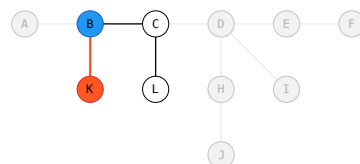
Шаг 5. $J \rightarrow$ код: H



Шаг 6. $H \rightarrow$ код: D



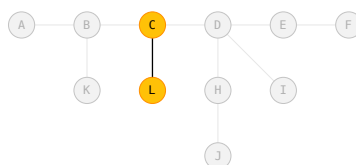
Шаг 7. $D \rightarrow$ код: C



Шаг 8. $K \rightarrow$ код: B



Шаг 9. $B \rightarrow$ код: C



Шаг 10. Остались C и L — стоп

Алгоритм 10. Алгоритм Прима

Инициализация: $\text{start} \in V$ (любая вершина); дерево $T = \{\text{start}\}$.

Step: среди всех рёбер, соединяющих уже выбранную вершину с ещё не выбранной, берём ребро e с минимальным весом $\sigma(e)$; добавляем его (и новую вершину) в T . Новая вершина присоединяется не обязательно к последней — к *любой* из уже выбранных. Повторяем, пока T не охватит все вершины ($n - 1$ ребро).

Пример (старт из J):

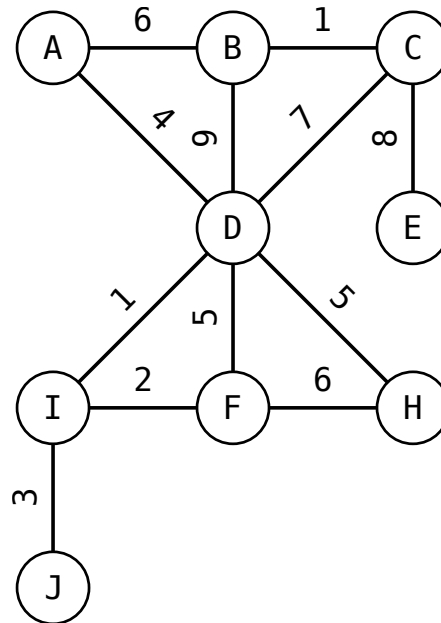
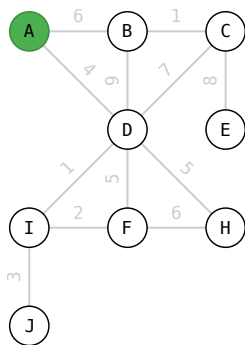


Рис. 6. Алг. Прима: взвешенный граф 9 вершин

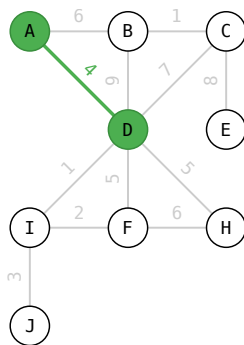
V	e	$\sum \sigma(e)$	комментарий
J	—	0	старт
I	$IJ(3)$	3	единственное ребро из J
D	$DI(1)$	4	$DI(1) < IF(2)$
F	$IF(2)$	6	$IF(2) < AD(4) < DH(5) = DF(5) < \dots$
A	$AD(4)$	10	$AD(4) < DH(5) < FH(6) < \dots$
H	$DH(5)$	15	$DH(5) < AB(6) = FH(6) < DC(7)$
B	$AB(6)$	21	$AB(6) < DC(7) < BD(9)$
C	$BC(1)$	22	после прихода B открылось ребро $BC(1)$
E	$CE(8)$	30	единственное ребро к последней вершине E

Минимальное остовное дерево: $IJ, DI, IF, AD, DH, AB, BC, CE$; суммарный вес $\sum \sigma = 30$.

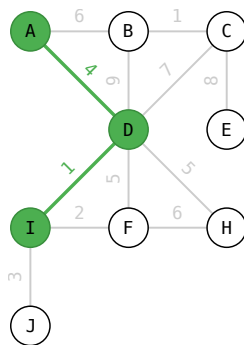
Работа алгоритма по шагам (выбранные рёбра — зелёные, текущее добавляемое — оранжевое, недоступные пока рёбра — серые):



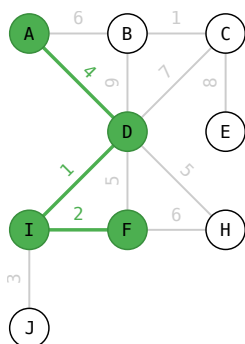
Шаг 0. Старт: $T = \{J\}$



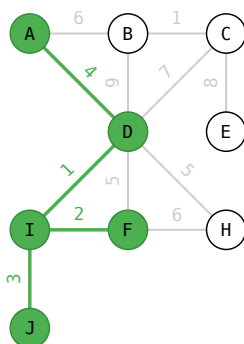
Шаг 1. $+IJ(3)$, $\Sigma = 3$



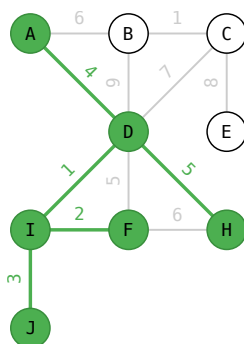
Шаг 2. $+DI(1)$, $\Sigma = 4$



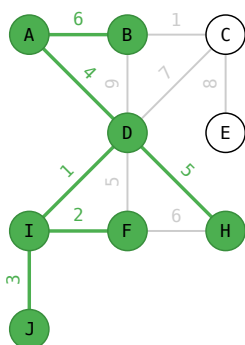
Шаг 3. $+IF(2)$, $\Sigma = 6$



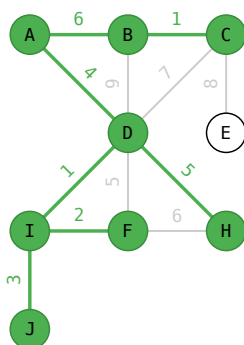
Шаг 4. $+AD(4)$, $\Sigma = 10$



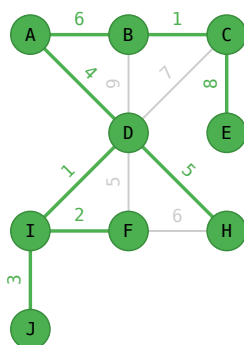
Шаг 5. $+DH(5)$, $\Sigma = 15$



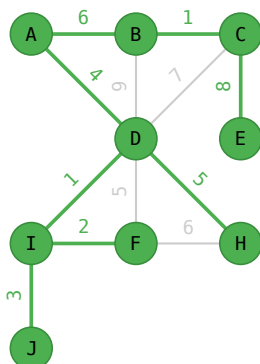
Шаг 6. $+AB(6)$, $\Sigma = 21$



Шаг 7. $+BC(1)$, $\Sigma = 22$



Шаг 8. $+CE(8)$, $\Sigma = 30$



Шаг 9. Остов построен, $\Sigma = 30$

Замечание. Если все веса рёбер различны, минимальное остовное дерево единственно — и алгоритм Прима находит именно его независимо от выбора стартовой вершины.

Алгоритм 11. Алгоритм Краскала

Step: рёбра перебираются в порядке возрастания веса; очередное ребро e с минимальным $\sigma(e)$ добавляется в остов, если при этом *не появляется цикл* (т. е. концы e лежат в разных компонентах уже построенного леса); иначе ребро пропускается.

Пример (граф той же структуры, что у алгоритма Прима, с другими весами):

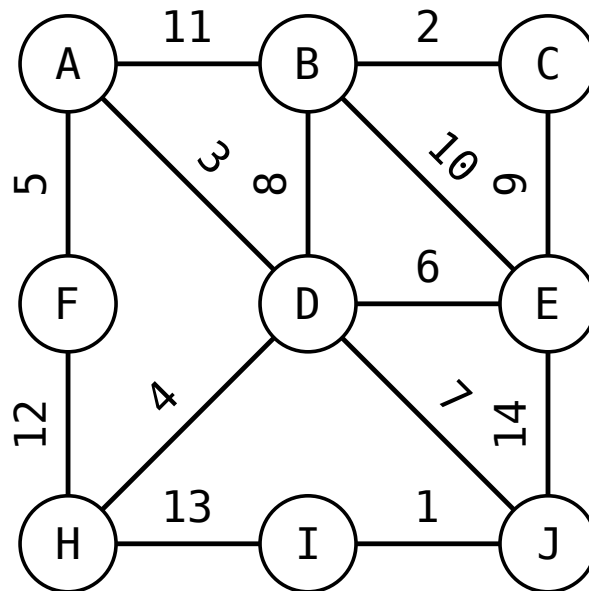


Рис. 7. Алг. Краскала: взвешенный граф 9 вершин

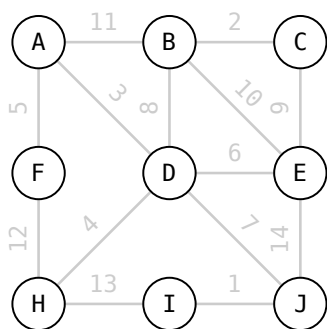
V	e	$\sum \sigma(e)$	комментарий
—	—	0	лес из 9 изолированных вершин
J, I	$JI(1)$	1	соединяем $\{J\}$ и $\{I\}$
B, C	$BC(2)$	3	соединяем $\{B\}$ и $\{C\}$
A, D	$AD(3)$	6	соединяем $\{A\}$ и $\{D\}$
H	$HD(4)$	10	H присоединяется к $\{A, D\}$
F	$AF(5)$	15	F присоединяется к $\{A, D, H\}$
E	$DE(6)$	21	E присоединяется к $\{A, D, H, F\}$
—	$JD(7)$	28	сливаем $\{J, I\}$ и $\{A, D, H, F, E\}$
—	$BD(8)$	36	сливаем $\{B, C\}$ с остальными; 8 рёбер — стоп

Оставшиеся рёбра $EC(9)$, $BE(10)$, $AB(11)$, $FH(12)$, $HI(13)$, $EJ(14)$ замыкали бы циклы и пропускаются. Минимальное остовное дерево: $JI, BC, AD, HD, AF, DE, JD, BD$, суммарный вес $\sum \sigma = 36$.

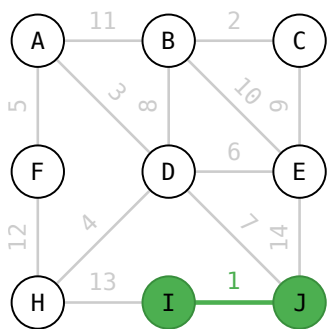
Упр. Почему алгоритм сойдётся (построит остов)?

Ответ. Каждое добавленное ребро уменьшает число компонент леса ровно на 1: начали с $|V|$ компонент, дерево получится после $|V| - 1$ добавлений. Застрять алгоритм не может: пока компонент больше одной, в связном графе обязательно найдётся ребро между разными компонентами — и при просмотре списка оно не будет пропущено.

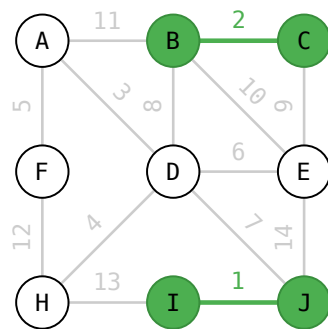
Работа алгоритма по шагам (добавленные рёбра — зелёные, текущее — оранжевое, пропущенные — бледно-серые):



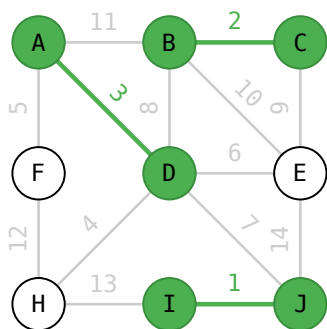
Шаг 0. Лес из изолированных вершин



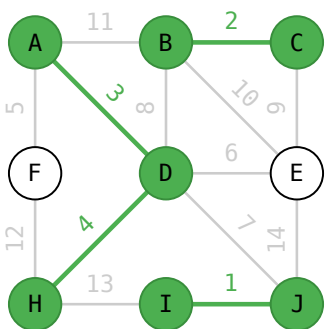
Шаг 1. $+JI(1)$



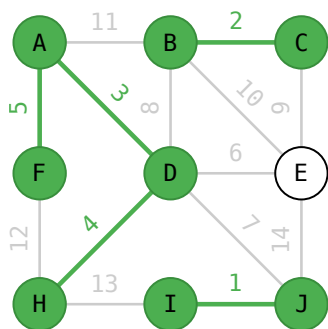
Шаг 2. $+BC(2)$



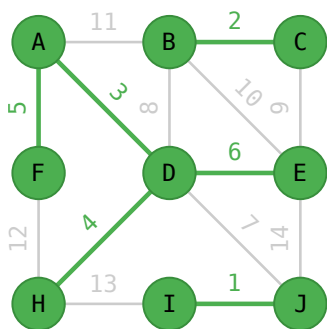
Шаг 3. $+AD(3)$



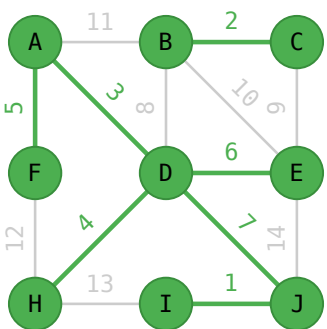
Шаг 4. $+HD(4)$



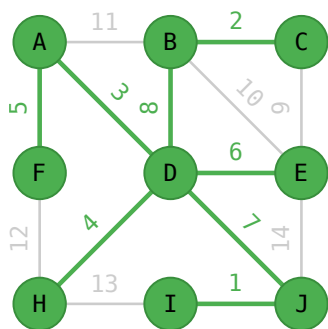
Шаг 5. $+AF(5)$



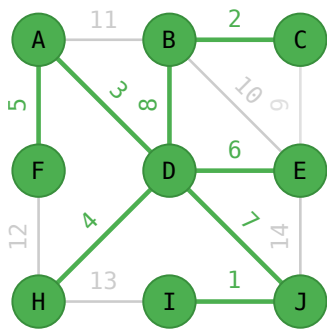
Шаг 6. $+DE(6)$



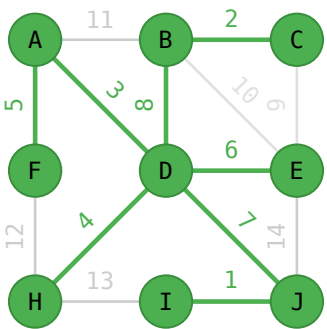
Шаг 7. $+JD(7)$



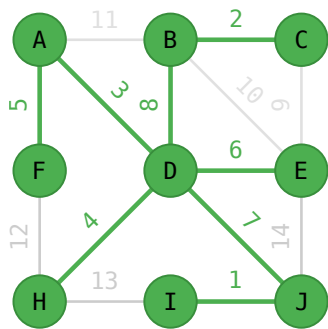
Шаг 8. $+BD(8)$: остов готов



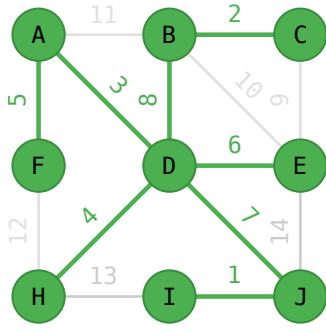
Шаг 9. $EC(9)$ — цикл, пропуск



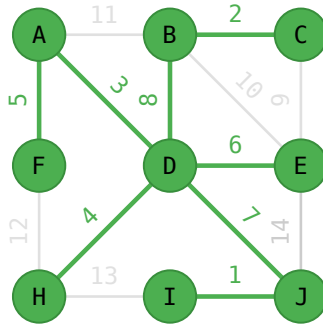
Шаг 10. $BE(10)$ — пропуск



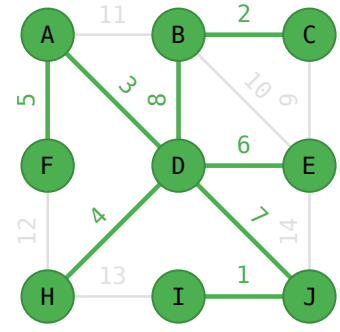
Шаг 11. $AB(11)$ — пропуск



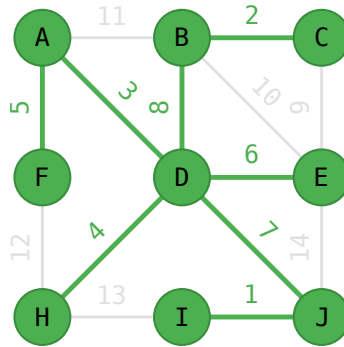
Шаг 12. $FH(12)$ — пропуск



Шаг 13. $HI(13)$ — пропуск



Шаг 14. $EJ(14)$ — пропуск



Шаг 15. Остов построен, $\Sigma = 36$

Алгоритм 12. Алгоритм поиска наибольшего паросочетания

Дано: $G(V, E)$ — двудольный, $V = V_1 \sqcup V_2$.

Найти: наибольшее паросочетание $\hat{E}$.

Инициализация. Добавляем фиктивные вершины S и F :

$$\forall v_i \in V_1: \text{ дуга } S \rightarrow v_i; \quad \forall w_j \in V_2: \text{ дуга } w_j \rightarrow F,$$

а все рёбра графа ориентируем слева направо: $v_i \rightarrow w_j$.

Step. Пока существует путь $l: S \rightarrow \dots \rightarrow F$:

1. находим такой путь (любым обходом);
2. *инвертируем* на нём все рёбра между долями: пройденное ребро $v_i \rightarrow w_j$ разворачивается в $w_j \rightarrow v_i$ — развёрнутые рёбра и есть текущее паросочетание;
3. крайние дуги пути ($S \rightarrow v_i$ и $w_j \rightarrow F$) удаляем — вершины v_i и w_j насыщены.

Когда путей из S в F больше нет, развёрнутые рёбра $w_j \rightarrow v_i$ образуют наибольшее паросочетание.

Почему это работает: путь $S \rightarrow v_i \rightarrow w_j \rightarrow v_k \rightarrow \dots \rightarrow F$, проходящий по развёрнутому ребру $w_j \rightarrow v_k$, — это *увеличивающий путь*: он отбирает w_j у вершины v_k и тут же находит ей новую пару, а в конце насыщает одну новую вершину каждой доли. Число рёбер паросочетания каждый раз растёт на 1.

Пример. $V_1 = \{A, B, C, D\}$, $V_2 = \{\alpha, \beta, \gamma\}$, рёбра: $A\alpha, A\beta, B\alpha, B\beta, C\beta, C\gamma, D\alpha, D\gamma$.

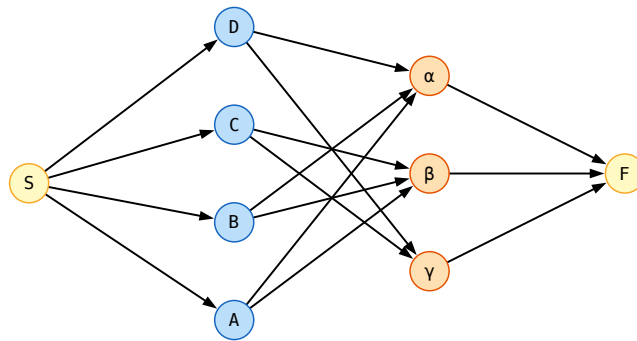


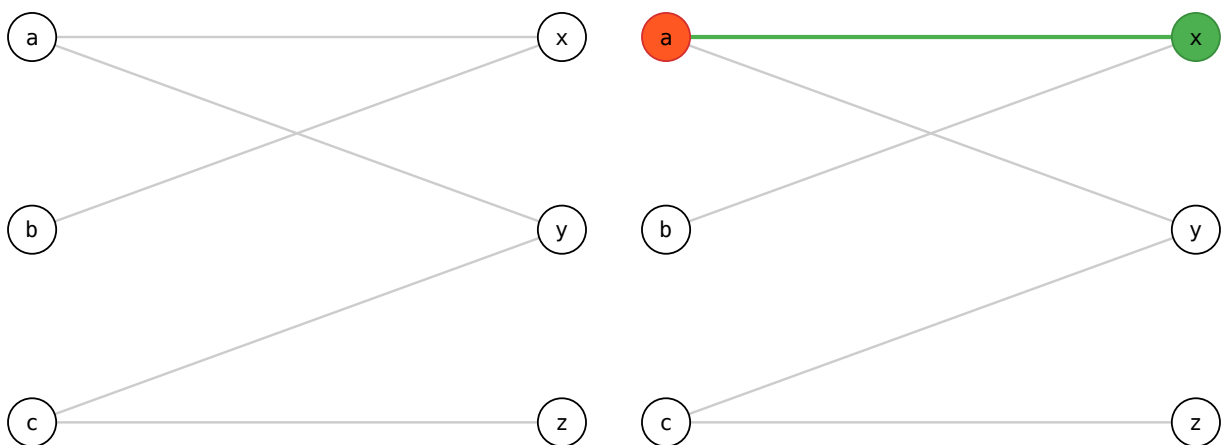
Рис. 8. Алг. наиб. паросочетания: S , доли $\{A, B, C, D\}$ и $\{\alpha, \beta, \gamma\}$, F

Находим пути и инвертируем (стрелки «туда-обратно»):

1. S, B, β, F — ребро $B\beta$ разворачивается: $\beta \rightarrow B$; паросочетание $\{B\beta\}$;
2. $S, C, \beta, B, \alpha, F$ — путь идёт по развёрнутому ребру $\beta \rightarrow B$ (отбираем β у B): теперь $\beta \rightarrow C$, $\alpha \rightarrow B$; паросочетание $\{C\beta, B\alpha\}$;
3. $S, A, \beta, C, \gamma, F$ — аналогично β переходит к A , а C получает γ ; паросочетание $\{A\beta, B\alpha, C\gamma\}$.

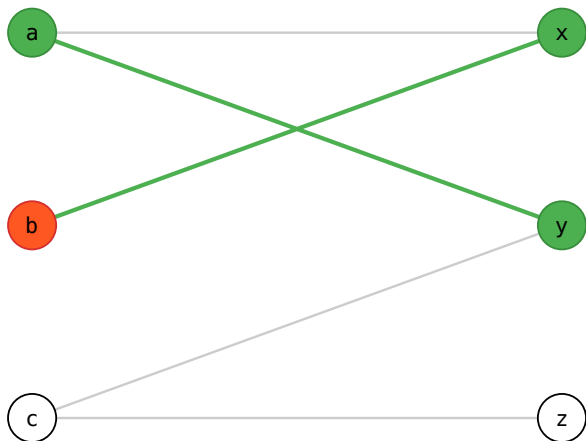
Итог: наибольшее паросочетание $\hat{E} = \{A\beta, B\alpha, C\gamma\}$, $|\hat{E}| = 3 = \min(|V_1|, |V_2|)$ — вершина D осталась свободной.

Работа алгоритма по шагам (текущий путь — оранжевый, развёрнутые рёбра паросочетания — зелёные, удалённые дуги — бледные):

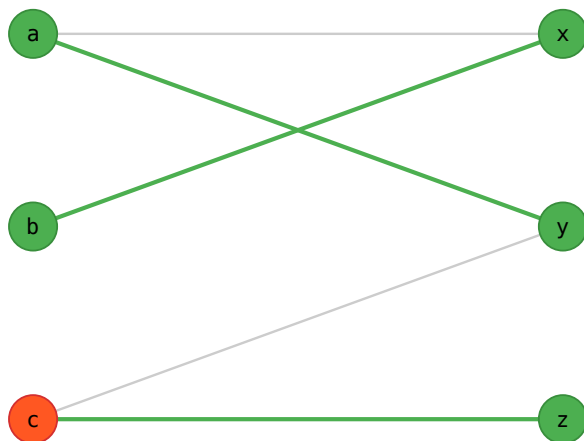


Шаг 0. Сеть с фиктивными S и F

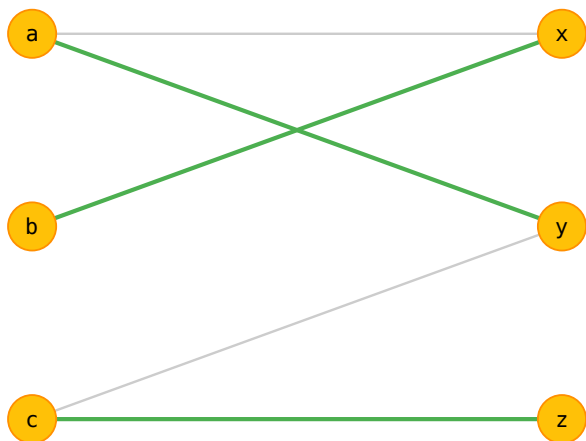
Шаг 1. Путь 1: S, B, β, F



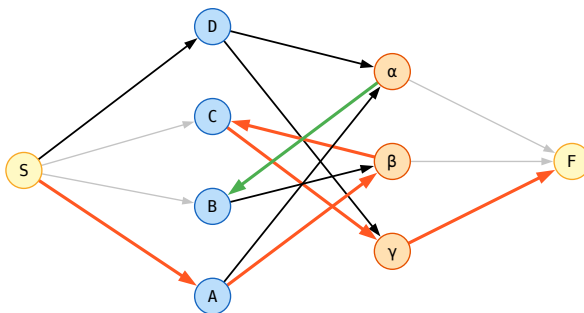
Шаг 2. $B\beta$ развёрнуто: $\{B\beta\}$



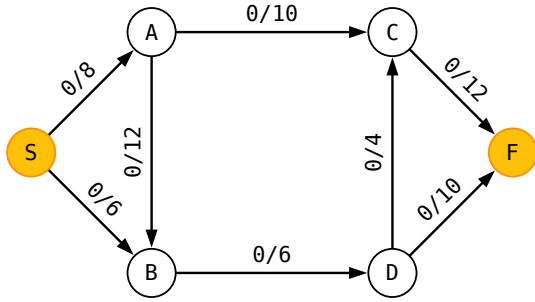
Шаг 3. Путь 2: $S, C, \beta, B, \alpha, F$



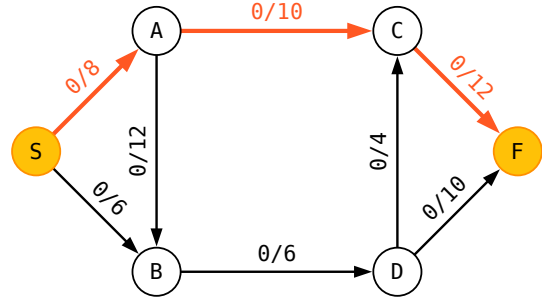
Шаг 4. β перешла к C , B получил α



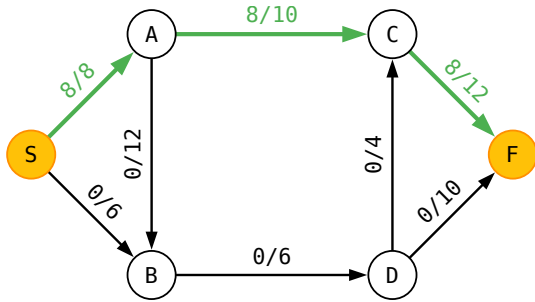
Шаг 5. Путь 3: $S, A, \beta, C, \gamma, F$



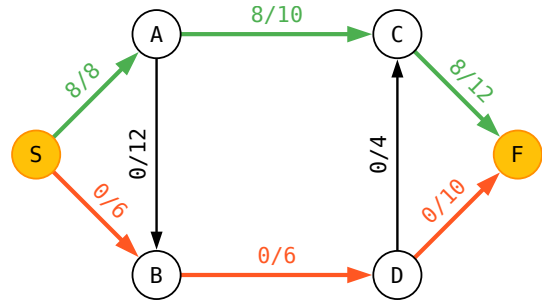
Шаг 0. Инициализация: $f \equiv 0$, $\tilde{\sigma} = \sigma$



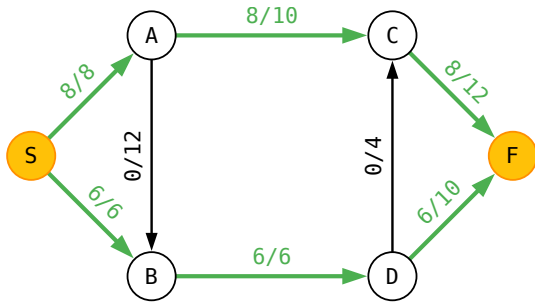
Шаг 1. Путь $S \rightarrow A \rightarrow C \rightarrow F$, $\delta = \min(2, 1, 3) = 1$



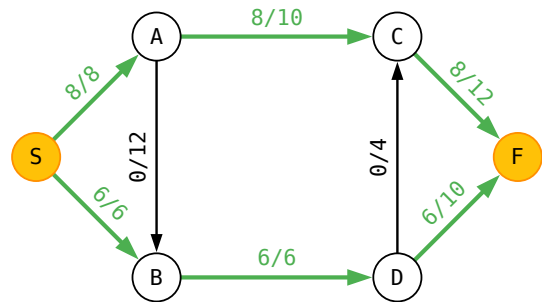
Шаг 2. Поток проведён: $|f| = 1$; $A \rightarrow C$ исчерпано



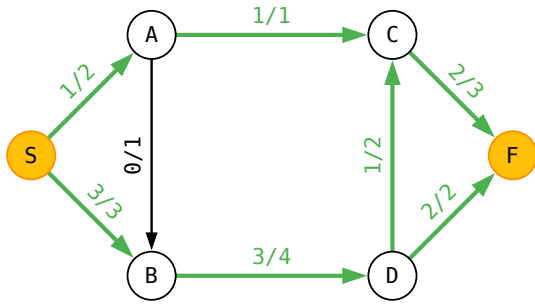
Шаг 3. Путь $S \rightarrow B \rightarrow D \rightarrow F$, $\delta = \min(3, 4, 2) = 2$



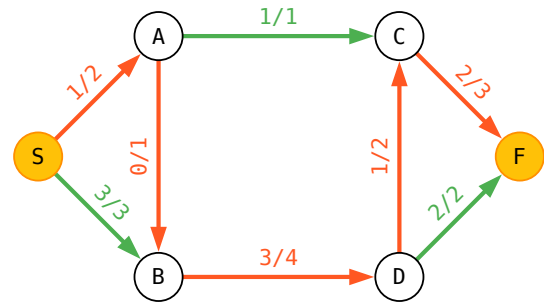
Шаг 4. $|f| = 3$; ребро $D \rightarrow F$ исчерпано



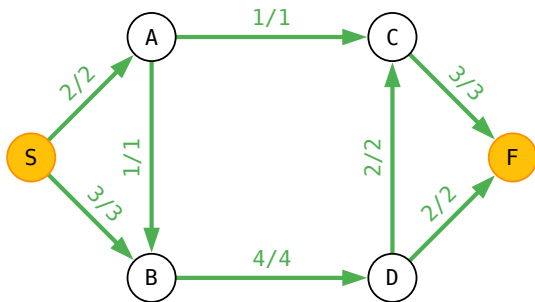
Шаг 5. Путь $S \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



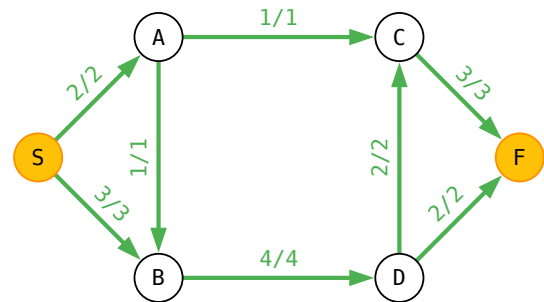
Шаг 6. $|f| = 4$; ребро $S \rightarrow B$ исчерпано



Шаг 7. Путь $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow F$, $\delta = 1$



Шаг 8. $|f| = 5$; ребро $S \rightarrow A$ исчерпано



Шаг 9. Путь $S \rightarrow F$ в остаточной сети нет: $|f|_{\max} = 5$

Комментарии к шагам. На каждом шаге величина потока увеличивается на узкое место δ найденного пути: $1 + 2 + 1 + 1 = 5$. После шага 8 оба ребра из истока ($S \rightarrow A$ и $S \rightarrow B$) исчерпаны: в остаточной сети из S не выходит ни одного ребра, дополняющих путей нет — алгоритм останавливается.

Проверка через разрез. Набор рёбер $\tilde{E} = \{S \rightarrow A, S \rightarrow B\}$ — главный разрез ($\{S\}$ против остальных вершин) с пропускной способностью $\sigma(\tilde{E}) = 2 + 3 = 5$. Мощность любого потока не превосходит пропускной способности любого разреза, поэтому $|f| = 5 = \sigma(\tilde{E})$ — поток максимален, а разрез минимален (теорема Форда–Фалкерсона: $\max |f| = \min \sigma(\tilde{E})$).

Алгоритм 14. Метод проталкивания предпотока (Push-Relabel)

Инициализация:

1. $h(S) := |V|$; $\forall u \in V \setminus \{S\}: h(u) := 0$.
2. $\tilde{f}_{\text{start}}: \forall v \in \text{Ch}(S): \tilde{f}(S, v) := \sigma(\overline{Sv}), \quad \tilde{f}(v, S) := -\sigma(\overline{Sv});$
 $\forall u, v \in V \setminus \{S\}: \tilde{f}(u, v) := 0$ (рёбра из истока насыщаются полностью).

Основной цикл: Пока $\exists v \in V \setminus \{S, F\}$ т. ч. $\varepsilon(v) > 0$ (*активная* вершина):

Push (проталкивание): если $\exists u \in \text{Ch}(v)$ (в остаточной сети) т. ч. $h(u) < h(v)$:

$$\delta_{\text{new}} := \min\{\varepsilon(v), \tilde{\sigma}(\overline{vu})\},$$

$$\tilde{f}_{\text{new}}(v, u) := \tilde{f}_{\text{old}}(v, u) + \delta_{\text{new}}, \quad \tilde{f}_{\text{new}}(u, v) := -\tilde{f}_{\text{new}}(v, u),$$

$$\tilde{\sigma}_{\text{new}}(v, u) := \tilde{\sigma}_{\text{old}}(v, u) - \delta_{\text{new}}, \quad \tilde{\sigma}_{\text{new}}(u, v) := \tilde{\sigma}_{\text{old}}(u, v) + \delta_{\text{new}}.$$

Здесь $\tilde{\sigma}(u, v) + \tilde{\sigma}(v, u) = \sigma(v, u)$ для $vu = e$ (остаточные пропускные способности, как в алгоритме Форда–Фалкерсона; проталкивание по обратному ребру означает *возврат* ранее протолкнутого потока).

Lift (подъём): если $\nexists u \in \text{Ch}(v)$ т. ч. $h(u) < h(v)$, то

$$h(v) := \min_{u \in \text{Ch}(v)} h(u) + 1$$

(минимум — по соседям в остаточной сети; после подъёма проталкивание из v снова возможно).

Выход: когда активных вершин не осталось, предпоток стал потоком (все внутренние эксцессы равны нулю), и этот поток — максимальный; $|f| = \varepsilon(F)$.

Пример. Сеть S, A, B, C, D, F с пропускными способностями (та же топология, что в лекциях 9–10):

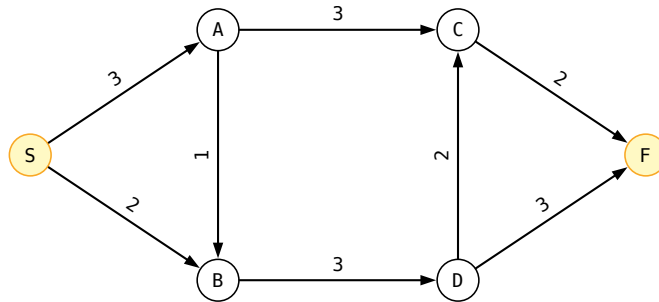


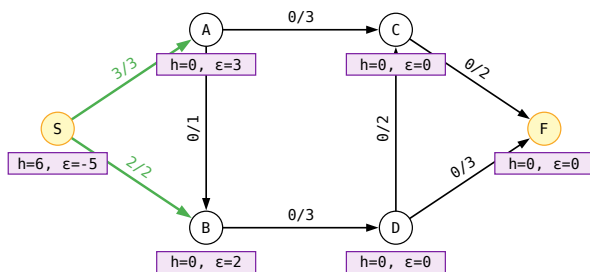
Рис. 9. Сеть Push-Relabel: S, A, B, C, D, F , σ на рёбрах

Протокол работы (каждая строка — одна операция; δ — объём проталкивания):

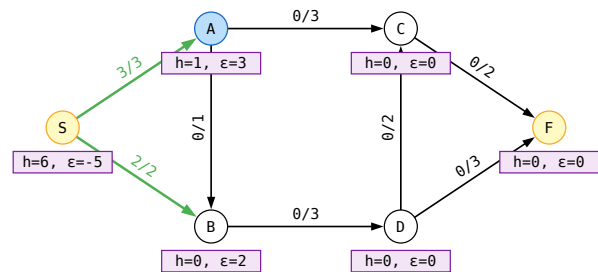
№	операция	δ	комментарий
0	инициализация	—	$S \rightarrow A, S \rightarrow B$ насыщены: $\varepsilon(A)=3, \varepsilon(B)=2$
1	lift $A: h(A):=1$	—	соседи A имеют $h = 0$, проталкивать некуда
2	push $A \rightarrow C$	3	$h(C) = 0 < 1; \varepsilon(A)=0, \varepsilon(C)=3$
3	lift $B: h(B):=1$	—	
4	push $B \rightarrow D$	2	$\varepsilon(B)=0, \varepsilon(D)=2$
5	lift $C: h(C):=1$	—	
6	push $C \rightarrow F$	2	ребро $C \rightarrow F$ насыщено; $\varepsilon(C)=1$
7	lift $C: h(C):=2$	—	вниз идти некуда: остаточный сосед — только A ($h=1$)
8	push $C \rightarrow A$	1	возврат по обратному ребру: $A \rightarrow C$ теперь $2/3; \varepsilon(A)=1$
9	lift $A: h(A):=2$	—	остаточные соседи: C ($h=2$), B ($h=1$), S ($h=6$)
10	push $A \rightarrow B$	1	$h(B) = 1 < 2; \varepsilon(B)=1$
11	push $B \rightarrow D$	1	$\varepsilon(D)=3$
12	lift $D: h(D):=1$	—	сосед F имеет $h = 0$
13	push $D \rightarrow F$	3	$\varepsilon(D)=0$; активных вершин нет — стоп

Итог: $|f| = \varepsilon(F) = 2 + 3 = 5$; поток по рёбрам: $S \rightarrow A = 3, S \rightarrow B = 2, A \rightarrow C = 2, A \rightarrow B = 1, B \rightarrow D = 3, C \rightarrow F = 2, D \rightarrow F = 3, D \rightarrow C = 0$ — условие балансировки выполнено во всех внутренних вершинах.

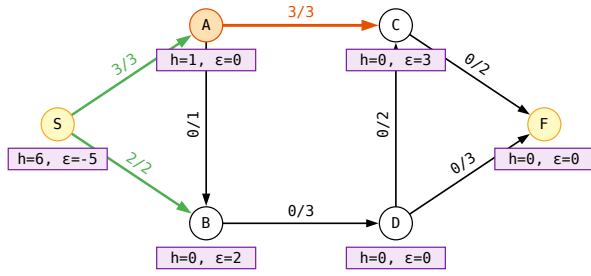
Работа алгоритма по шагам. Подпись ребра — $\tilde{f}/\sigma$; под каждой вершиной — её h и ε . Активная вершина операции — оранжевая, поднимаемая — синяя, ребро проталкивания — оранжевое, рёбра с положительным потоком — зелёные.



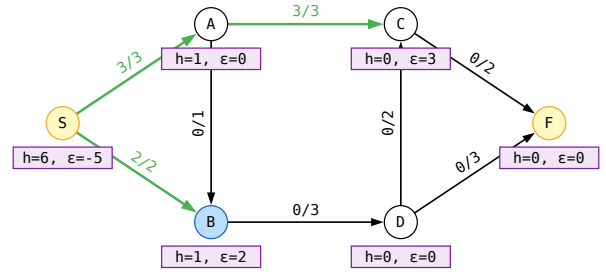
Шаг 0. Инициализация: рёбра из S насыщены



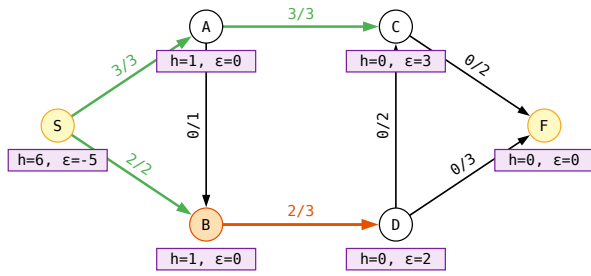
Шаг 1. Lift $A: h(A) := 1$



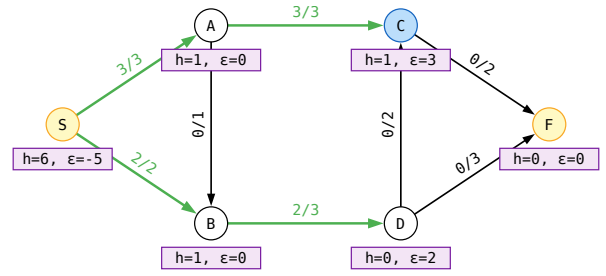
Шаг 2. Push $A \rightarrow C$, $\delta = 3$



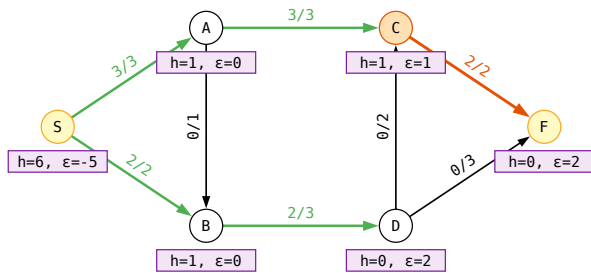
Шаг 3. Lift B : $h(B) := 1$



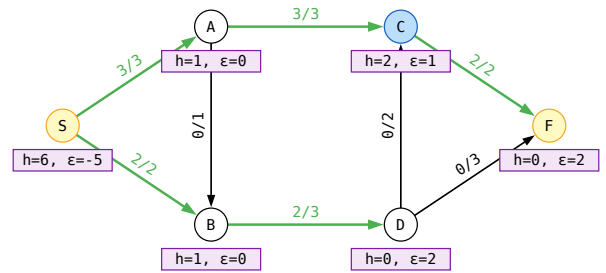
Шаг 4. Push $B \rightarrow D$, $\delta = 2$



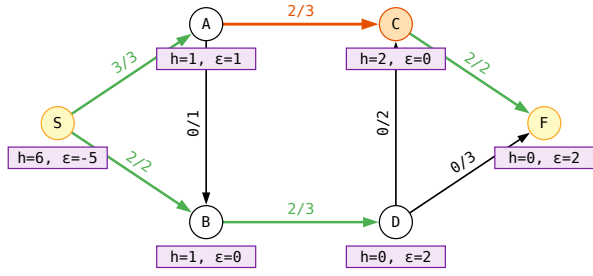
Шаг 5. Lift C : $h(C) := 1$



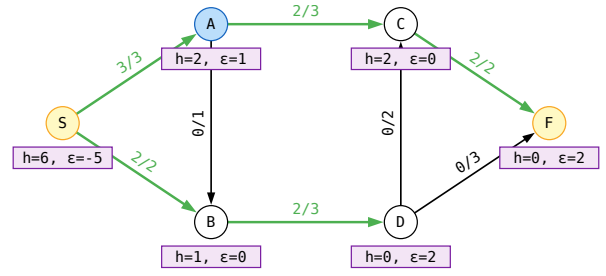
Шаг 6. Push $C \rightarrow F$, $\delta = 2$: ребро насыщено



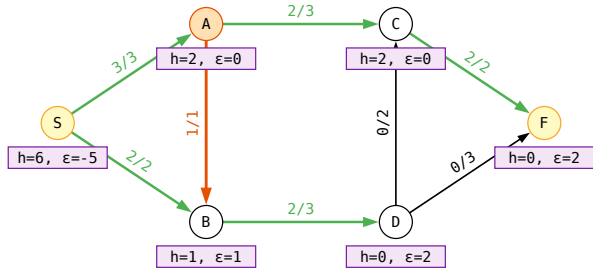
Шаг 7. Lift C : $h(C) := 2$ (выше A)



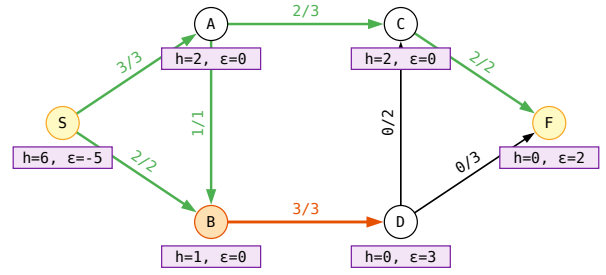
Шаг 8. Push $C \rightarrow A$, $\delta = 1$: возврат, $A \rightarrow C$ стало $2/3$



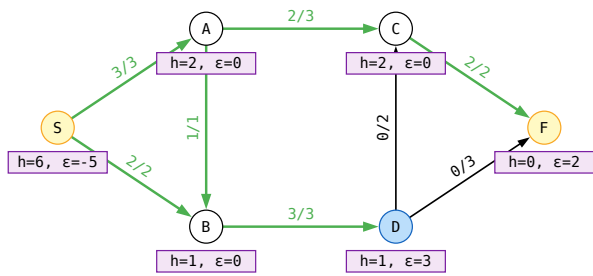
Шаг 9. Lift A: $h(A) := 2$



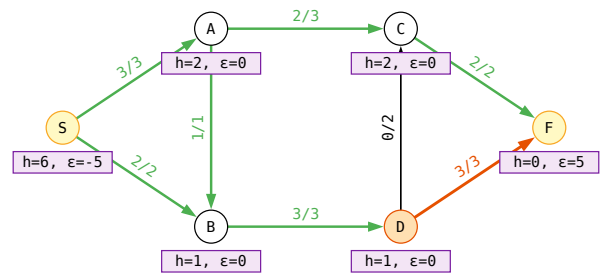
Шаг 10. Push $A \rightarrow B$, $\delta = 1$



Шаг 11. Push $B \rightarrow D$, $\delta = 1$



Шаг 12. Lift D: $h(D) := 1$



Шаг 13. Push $D \rightarrow F$, $\delta = 3$: $|f| = 5$, конец

Алгоритм 15. Алгоритм Дейкстры

Требование: $\sigma: E \rightarrow \mathbb{R}_{\geq 0}$ (только неотрицательные веса!).

Инициализация: $\mu(\text{start}) := 0$, $\mu(v) := \infty$ для всех остальных вершин.

Шаг: Выбираем v с минимальной меткой $\mu(v)$ среди непосещённых.

Для каждого $w \in \text{Ch}(v)$:

$$\text{если } \mu(w) > \mu(v) + \sigma(\overline{vw}), \text{ то } \begin{cases} \mu(w) := \mu(v) + \sigma(\overline{vw}), \\ P(w) := v. \end{cases}$$

Это **динамическое программирование** (жадный выбор + оптимальная подструктура).

Почему жадный выбор корректен: пусть v — непосещённая вершина с минимальной меткой $\mu(v)$. Любой путь из старта в v обязан в какой-то момент впервые покинуть множество посещённых вершин по ребру $x \rightarrow y$ (y не посещена); его длина не меньше $\mu(y) \geq \mu(v)$, а дальнейшие рёбра *неотрицательны* — короче $\mu(v)$ не получится. Значит, метку $\mu(v)$ можно зафиксировать как окончательную. (Именно здесь существенно $\sigma \geq 0$: с отрицательными рёбрами «дешёвый обход позже» возможен, и жадность ломается — нужен Форд–Беллман.)

Пример (рис. p29-dijkstra-example-1): граф A, B, C, D . Start = A . Рёбра: BA (2), CB (5), AC (10), DC (3), AD (2), BD (7).

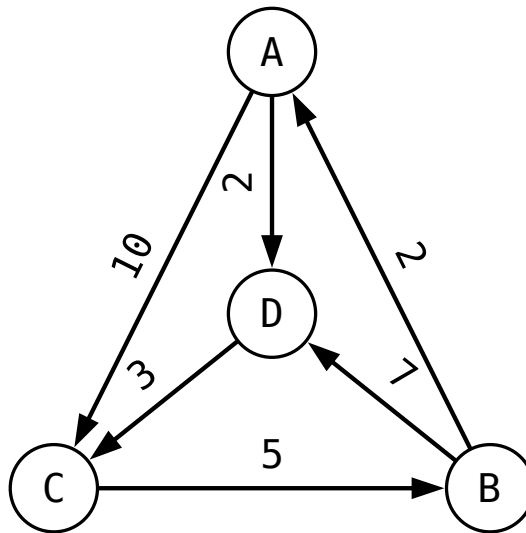


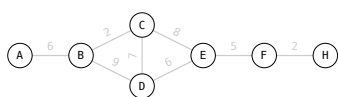
Рис. 10. Алг. Дейкстры: A, B, C, D ($AD=2, AC=10, DC=3, BD=7, CB=5, BA=2$)

	A	B	C	D
Иниц.	0	∞	∞	∞
A	—	∞	10, A	2, A
D	—	∞	5, D	—
C	—	10, C	—	—

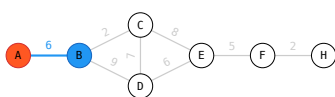
$D \rightarrow C$: $\mu(D) = 2$, $\sigma(DC) = 3$, итого $2 + 3 = 5 < 10$, обновляем.

Дерево кратчайших путей: $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$.

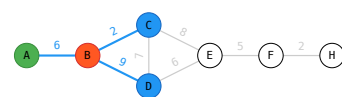
Работа алгоритма по шагам. Для разнообразия демонстрация выполнена на другом графе (7 вершин, рис. p9-example-0, веса проставлены детерминированно), старт — A .



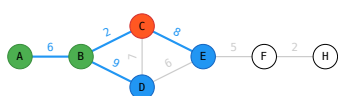
Шаг 0



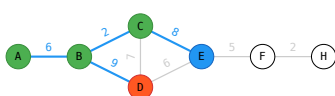
Шаг 1



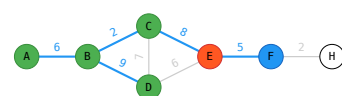
Шаг 2



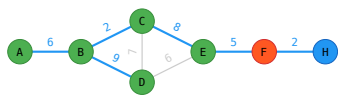
Шаг 3



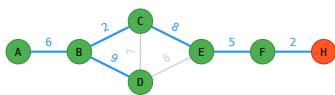
Шаг 4



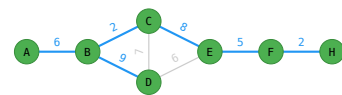
Шаг 5



Шаг 6



Шаг 7



Шаг 8

Алгоритм 16. Алгоритм Форда-Беллмана

Допускается: $\sigma: E \rightarrow \mathbb{R}$ (в том числе отрицательные веса).

Инициализация: Произвольное упорядочивание рёбер. Стартовая вершина v : $\mu(v) := 0$, $\mu(w) := \infty$ для остальных.

Шаг (1 шаг = полный просмотр списка рёбер):

$\forall e_i \in \text{list}(E)$, $e_i = uv$:

$$\text{если } \mu(v) > \mu(u) + \sigma(e_i), \text{ то } \begin{cases} \mu(v) := \mu(u) + \sigma(e_i), \\ P(v) := u. \end{cases}$$

Конец алгоритма: шаги k и $k + 1$ дают одинаковый результат. Если n вершин, то алгоритм сойдётся не позже чем за $n - 1$ шагов.

Почему работает: индукцией по k проверяется инвариант — после k -го прохода по списку рёбер метка $\mu(w)$ не превосходит длины кратчайшего пути из старта в w , содержащего $\leq k$ рёбер (релаксация последнего ребра такого пути произойдёт не позже k -го прохода). Кратчайшие пути просты, т.е. содержат не более $n - 1$ ребра, поэтому $n - 1$ проходов

достаточно. Если же обновления случаются и на n -м проходе — в графе есть достижимый отрицательный цикл (так его и детектируют).

Пример (рис. p30-fb-example-1): граф A, B, C, D . $\text{Start} = B$. Отличие от Дейкстры: ребро BA имеет вес -2 .

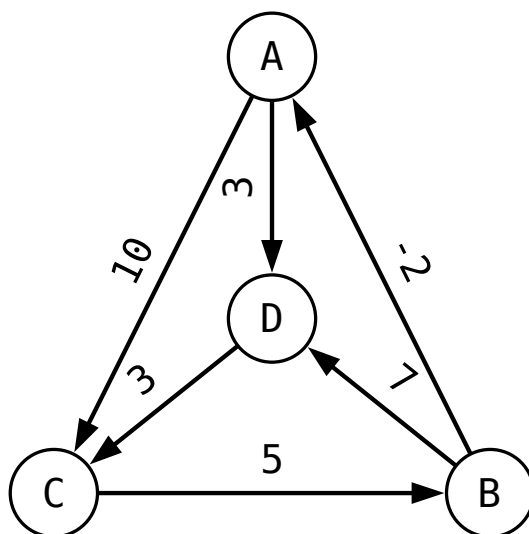


Рис. 11. Алг. Форда-Беллмана: тот же граф, $BA = -2$

Список рёбер с весами:

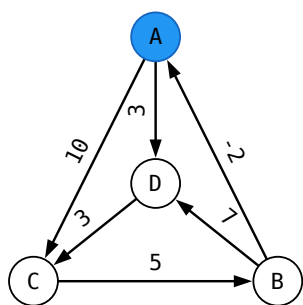
Ребро	Вес
AC	10
AD	2
BA	-2
BD	7
CB	5
DC	3

Протокол:

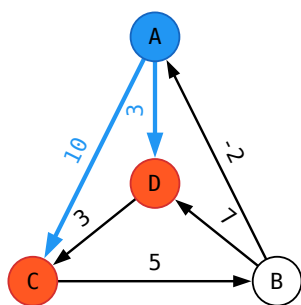
	A	B	C	D
Иниц.	∞	0	∞	∞
Шаг 1	-2, B	0	$7 + 3 = 10, D$	7, B
Шаг 2	-2, B	0	8, $A \rightarrow 3, D$	0, A

Корневое дерево кратчайших путей: $B \xrightarrow{-2} A \xrightarrow{2} D \xrightarrow{3} C$.

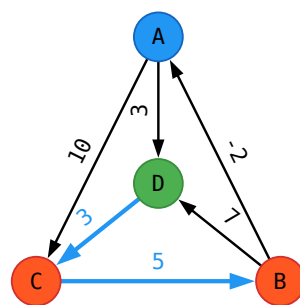
Работа алгоритма по шагам:



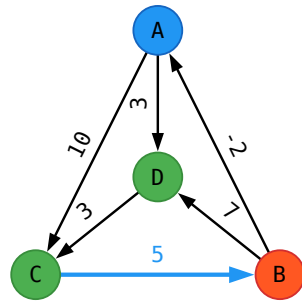
Шаг 0



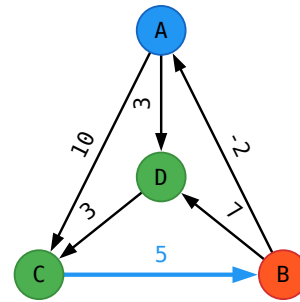
Шаг 1



Шаг 2



Шаг 3



Шаг 4

Алгоритм 17. Алгоритм Флойда-Уоршелла

Задача $\odot$: кратчайшие пути для каждой пары $u, v \in V$.

Идея: аналог алгоритма Уоршелла (транзитивное замыкание), но для взвешенного графа.

Инициализация: Матрица $\mu(u, w)$ — по весам рёбер. Диагональ: $\mu(v, v) = 0$. Нет рёбра: $\mu(u, w) = \infty$.

Шаг: v — ведущая вершина. На i -м шаге выделяем i -ю строку и i -й столбец и смотрим пересечения:

$\forall (u, w)$, где $u \neq v, w \neq v$:

$$\text{если } \mu(u, w) > \mu(u, v) + \mu(v, w), \text{ то } \begin{cases} \mu(u, w) := \mu(u, v) + \mu(v, w), \\ P(w) := \begin{cases} v, & \text{если } P(v) = \emptyset, \\ P(v), & \text{если } P(v) \neq \emptyset. \end{cases} \end{cases}$$

Пример (рис. p31-fu-example-1): тот же граф A, B, C, D .

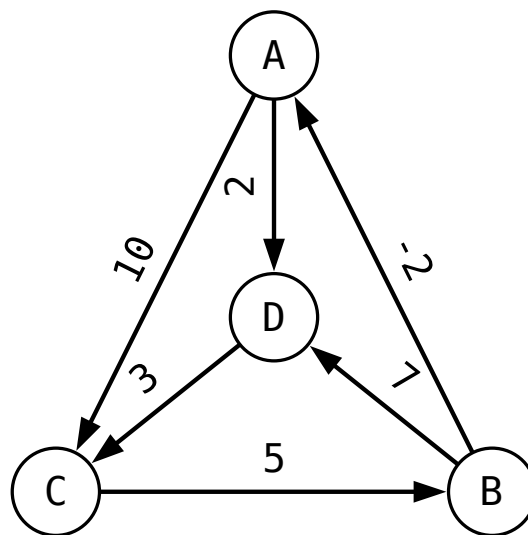


Рис. 12. Алг. Флойда-Уоршелла: тот же граф ($BA = -2$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	∞	10	2
B	-2	0	∞	7
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 1 (ведущая A): $\mu(B, C): \infty > (-2) + 10 = 8 (A)$; $\mu(B, D): 7 > (-2) + 2 = 0 (A)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	∞	5	0	∞
D	∞	∞	3	0

Шаг 2 (ведущая B): $\mu(C, A): \infty > 5 + (-2) = 3 (B)$; $\mu(C, D): \infty > 5 + 0 = 5 (B)$.

	A	B	C	D
A	0	∞	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	∞	∞	3	0

Шаг 3 (ведущая C): $\mu(A, B): \infty > 10 + 5 = 15 (C)$; $\mu(D, A): \infty > 3 + 3 = 6 (C)$; $\mu(D, B): \infty > 3 + 5 = 8 (C)$.

	A	B	C	D
A	0	15, C	10	2
B	-2	0	8, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Шаг 4 (ведущая D): $\mu(A, B): 15 > 2 + 8 = 10 (D)$ — путь $A \rightarrow D \rightarrow C \rightarrow B$; $\mu(A, C): 10 > 2 + 3 = 5 (D)$; $\mu(B, C): 8 > 0 + 3 = 3 (A)$ — путь $B \rightarrow A \rightarrow D \rightarrow C$, первая путевая точка A .

	A	B	C	D
A	0	10, D	5, D	2
B	-2	0	3, A	0, A
C	3, B	5	0	5, B
D	6, C	8, C	3	0

Восстановление: $A \rightarrow D$: путевая точка не записана (прямое ребро), путь $A \rightarrow D$, длина = 2. $C \rightarrow B$: путевая точка тоже не записана — кратчайший путь так и остался прямым ребром $C \rightarrow B$ длины 5 (обходы $C \rightarrow A \rightarrow \dots$ дороже: $\mu(C, A) = 3$, а ребра $A \rightarrow B$ нет, $\mu(A, B) = 10$). $A \rightarrow B$: $P_{AB} = D$, далее $\mu(D, B): P_{DB} = C$, далее прямое ребро $C \rightarrow B$; итог $A \xrightarrow{2} D \xrightarrow{3} C \xrightarrow{5} B$, длина = 10.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Алгоритм 18. Алгоритм Флойда

$G(V, E)$ — слабо связный ориентированный граф, $\sigma: E \rightarrow \mathbb{R}$.

Алгоритм аналогичен Флойду–Уоршеллу. На каждом шаге выделяем ведущую вершину (строку и столбец), проверяем все пересечения.

Хранение пути: для пары (u, w) храним P_{uw} — первую путевую точку. Для прямого ребра $A \rightarrow B$ — ничего хранить не надо. Остаток пути: $B \rightarrow \dots \rightarrow L$.

Если $\mu(u, w) > \mu(u, v) + \mu(v, w)$, то $\mu(u, w) := \mu(u, v) + \mu(v, w)$, $P_{uw} := P_{uv}$ (или v , если P_{uv} пуст).

Пример (рис. p32-f-example-1): граф A, B, C, D .

Список рёбер с весами:

Ребро	Вес
AB	1
BC	1
BD	3
CA	-2
CD	1
DA	4

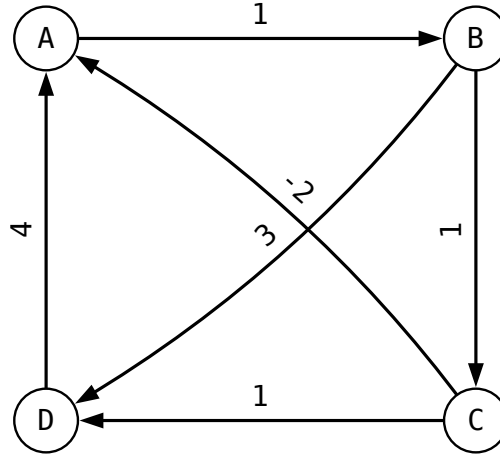


Рис. 13. Алг. Флойда: A, B, C, D ($AB=1$, $BC=1$, $BD=3$, $CA=-2$, $CD=1$, $DA=4$)

Шаг 0 (инициализация):

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	∞	0	1
D	4	∞	∞	0

Шаг 1 (ведущая A): выделяем строку A и столбец A . Обновления: $\mu(C, B): \infty > (-2) + 1 = -1$; $\mu(D, B): \infty > 4 + 1 = 5$.

	A	B	C	D
A	0	1	∞	∞
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	∞	0

Шаг 2 (ведущая B): обновления: $\mu(A, C): \infty > 1 + 1 = 2$; $\mu(A, D): \infty > 1 + 3 = 4$; $\mu(D, C): \infty > 5 + 1 = 6$.

	A	B	C	D
A	0	1	2, B	4, B
B	∞	0	1	3
C	-2	-1, A	0	1
D	4	5, A	6, A	0

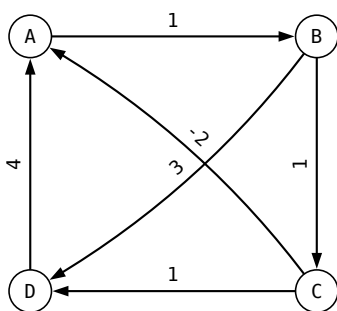
Шаг 3 (ведущая C): обновления: $\mu(A, D): 4 > 2 + 1 = 3$; $\mu(B, A): \infty > 1 + (-2) = -1$; $\mu(B, D): 3 > 1 + 1 = 2$.

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

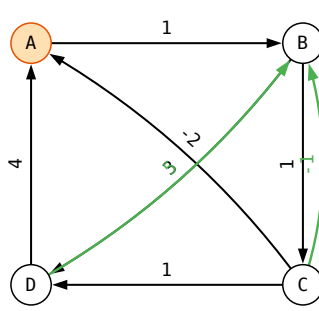
Шаг 4 (ведущая D): нет изменений (все пути через D дороже найденных).

	A	B	C	D
A	0	1	2, B	3, B
B	-1, C	0	1	2, C
C	-2	-1, A	0	1
D	4	5, A	6, A	0

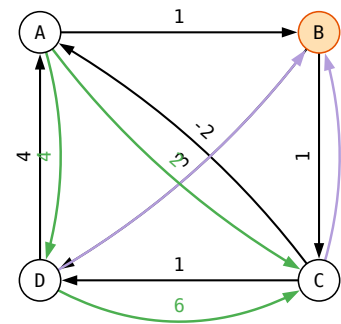
Работа алгоритма по шагам. Ведущая вершина — оранжевая; найденные на текущем шаге кратчайшие обходы — зелёные дуги с новой длиной, найденные ранее — светло-фиолетовые:



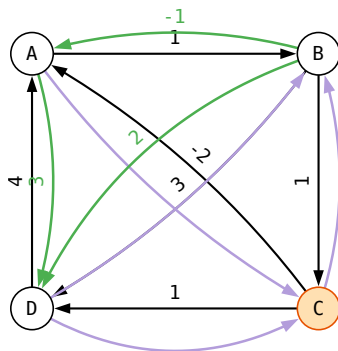
Шаг 0. Исходный граф



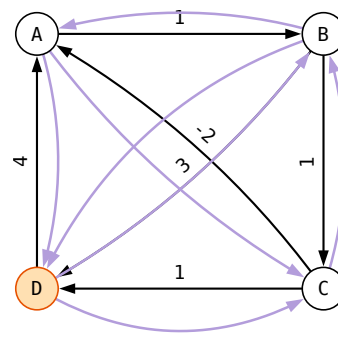
Шаг 1. Через A: $CB=-1$, $DB=5$



Шаг 2. Через B: $AC=2$, $AD=4$, $DC=6$



Шаг 3. Через C: $AD=3$, $BA=-1$, $BD=2$



Шаг 4. Через D обновлений нет — итог

Восстановление путей:

$A \rightarrow D$: $P_{AD} = B$. $A \xrightarrow{1} B$, $P_{BD} = C$, $B \xrightarrow{1} C$, P_{CD} пуст, $C \xrightarrow{1} D$. Итого: $A \xrightarrow{1} B \xrightarrow{1} C \xrightarrow{1} D$, длина = 3.

$C \rightarrow B$: $P_{CB} = A$. $C \xrightarrow{-2} A$, P_{AB} пуст, $A \xrightarrow{1} B$. Итого: $C \xrightarrow{-2} A \xrightarrow{1} B$, длина = -1.

Циклы отрицательного суммарного веса $\Leftrightarrow$ отрицательные значения на главной диагонали.

Почему: диагональный элемент $\mu(v, v)$ — длина кратчайшего замкнутого маршрута через v ; он станет отрицательным в точности тогда, когда через v проходит цикл отрицательного веса. Здесь диагональ нулевая: например, цикл $A \rightarrow B \rightarrow C \rightarrow A$ весит $1 + 1 - 2 = 0$, отрицательных циклов нет.

Алгоритм 19. Алгоритм Джонсона

Преобразование графа к виду с неотрицательными весами рёбер с сохранением набора кратчайших путей.

$G(V, E)$ — слабо связный, $\sigma: E \rightarrow \mathbb{R}$.

Идея. Дейкстра быстрее Флойда ($|V|$ запусков Дейкстры дешевле при разреженных графах), но не работает с отрицательными рёбрами. Джонсон «выпрямляет» веса потенциалами так, что все веса становятся неотрицательными, а кратчайшие пути не меняются.

Шаг 1. Джонсоновский потенциал.

Добавляем фиктивную вершину S : $\forall u \in V: e_u = Su, \sigma(e_u) = 0$.

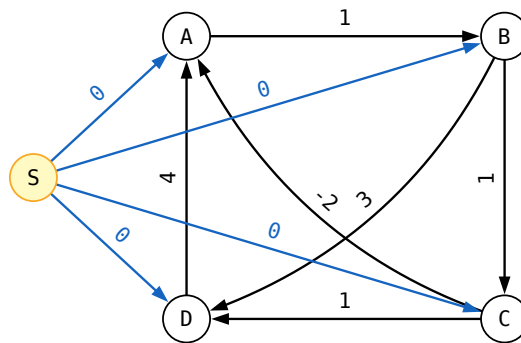


Рис. 14. Алг. Джонсона: тот же граф + фиктивная S (синие дуги веса 0)

Запускаем **алгоритм Форда–Беллмана** от S (Дейкстру пока нельзя — веса ещё отрицательные); $h(u) :=$ найденное расстояние от S до u .

Результат Форда–Беллмана (потенциалы):

$$h(A) = -2, \quad h(B) = -1, \quad h(C) = h(D) = 0.$$

(Проверка: до C и D дешевле всего прямые нулевые дуги из S , т.е. $h(C) = h(D) = 0$; до A выгоден путь $S \rightarrow C \rightarrow A$ весом $0 + (-2) = -2$; до B — путь $S \rightarrow C \rightarrow A \rightarrow B$ весом $0 - 2 + 1 = -1$.)

Шаг 2. Пересчёт весов.

Новая весовая функция:

$$\tilde{\sigma}(\overline{uv}) = \sigma(\overline{uv}) + h(u) - h(v) \geq 0.$$

Почему $\tilde{\sigma} \geq 0$: h — корректные расстояния от S , поэтому для любого ребра uv выполнено неравенство треугольника $h(v) \leq h(u) + \sigma(uv)$, что и означает $\sigma(uv) + h(u) - h(v) \geq 0$.

Ребро	σ	$\tilde{\sigma}$	Вычисление
AB	1	0	$1 + (-2) - (-1) = 0$
BC	1	0	$1 + (-1) - 0 = 0$
BD	3	2	$3 + (-1) - 0 = 2$
CA	-2	0	$-2 + 0 - (-2) = 0$
CD	1	1	$1 + 0 - 0 = 1$
DA	4	6	$4 + 0 - (-2) = 6$

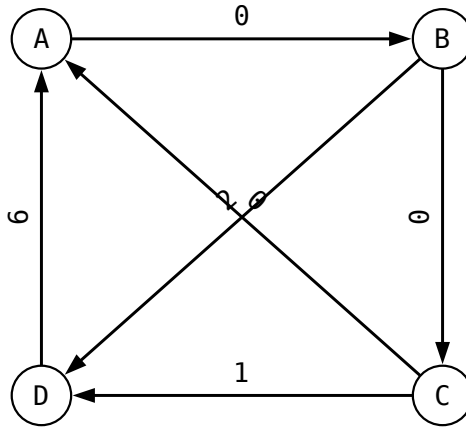


Рис. 15. Перевешенный граф: все $\tilde{\sigma} \geq 0$

Все $\tilde{\sigma} \geq 0$ — можно применять Дейкстру!

Шаг 3. Дейкстра от каждой вершины с весами $\tilde{\sigma}$.

Матрица расстояний в весах $\tilde{\sigma}$:

$\text{dist}_{\tilde{\sigma}}$	A	B	C	D
A	0	0	0	1
B	0	0	0	1
C	0	0	0	1
D	6	6	6	0

(Например, из D : единственный выход — дуга DA веса 6, дальше всё бесплатно по нулевым дугам $A \rightarrow B \rightarrow C$; поэтому вся строка D равна 6.)

Шаг 4. Обратный пересчёт.

Для получения истинных расстояний:

$$\text{dist}_{\sigma}(u, v) = \text{dist}_{\tilde{\sigma}}(u, v) - h(u) + h(v).$$

Почему пути не изменились: для любого пути $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$ его новая длина равна

$$\sum_i \tilde{\sigma}(x_i x_{i+1}) = \sum_i \sigma(x_i x_{i+1}) + h(u) - h(v)$$

— телескопическая сумма: все промежуточные потенциалы сокращаются. Все пути из u в v сдвинулись на *одну и ту же* константу $h(u) - h(v)$, поэтому порядок путей по длине (и сами кратчайшие пути) сохранился, а формула пересчёта очевидна.

Итоговая матрица истинных расстояний ($\text{dist}_{\sigma} = \text{dist}_{\tilde{\sigma}} - h(u) + h(v)$):

dist_{σ}	A	B	C	D
A	0	1	2	3
B	-1	0	1	2
C	-2	-1	0	1
D	4	5	6	0

Например, $\text{dist}_{\sigma}(D, A) = 6 - h(D) + h(A) = 6 - 0 - 2 = 4$; $\text{dist}_{\sigma}(B, A) = 0 - (-1) + (-2) = -1$. Матрица совпала с результатом алгоритма Флойда выше — проверка пройдена. ✓

Свойства:

1. Набор кратчайших путей для G в терминах σ и $\tilde{\sigma}$ совпадает (пути не меняются, только длины) — телескопическая сумма выше.

2. $\forall e \in E: \tilde{\sigma}(e) \geq 0$ (неравенство треугольника для корректных потенциалов).
3. Если $\tilde{\sigma}(e) < 0$ для какого-то ребра — значит, потенциалы найдены неправильно.